Is It Model Drift or a Broken Pipeline? What Re-Execution Can and Cannot Identify in Production ML

Ankitha Vuyyala
Independent Researcher, Louisville, Kentucky, USA
ankithavuyyala08@gmail.com
ORCID: https://orcid.org/0009-0004-1916-8070
October 2026

ABSTRACT
When a production model degrades, teams usually retrain it, although the model sees the output of a data pipeline that changes all the time. I ask when a pipeline defect can be told apart from a genuine change of the prediction problem at all. I prove three results. A defect whose corrupted model inputs stay in the image of the correct pipeline is invisible to every diagnostic that monitors model inputs and labels. Re-executing a changed component at its known-good version (a rollback test followed by a propagation test) decides exactly whether it is an effective defect whenever the data source did not force the change. When the source did force it, no re-execution can certify the rewritten component without evidence about the new encoding. These results draw an identifiability boundary. They yield a three-verdict rule that, under an accurate record of the source's changes, is both sound (it fixes only verified defects) and complete (it lists every component a published change touched for review, so no consequential defect is missed silently), and decides exactly where no change touched the pipeline. I tested these claims in four pre-registered rounds on a benchmark of eleven executable pipelines over public data with 2,961 evaluated cases built on dated changes of the published files, real library-upgrade behaviours and replays of public incidents, with ground truth from executing every action. Interventional diagnosis was correct on 267 of the 267 consequential cases without a published change in their period range. Where the source had changed, every method that commits to a diagnosis erred often, by blaming healthy components or by missing defects. In the fourth round, on two three-table pipelines of eleven and twelve components, my three-verdict diagnoser DoD-4 had a false-blame rate of 0.000 over 450 cases and a silent-miss rate of 0.000 over 256 consequential ones; it decided 0.11 of the cases (correct on 1.00 of those) and listed 5.3 components per case for review. What remains open is not detection but certifying source-forced adaptations, which needs evidence from the data provider rather than from the pipeline.

Keywords: machine learning in production; data pipelines; distribution shift; root-cause diagnosis; identifiability; interventional debugging; selective prediction; benchmark

1. Introduction

A production model's accuracy drops and an alarm fires. The default response is to retrain on recent data, and monitoring tools are built around it: they compare input and output distributions with a reference window and trigger the retraining pipeline. But a model's inputs are not the world; they are the output of a data pipeline that changes all the time. Studies of production ML report that most outages of a large ML pipeline were not ML-centric [1], data and pipeline problems are common [2], [3], and individual incidents show how costly silent pipeline faults are: a logging change at Uber removed a fare component for 10% of sessions in key US cities and went undetected for 45 days [4]. Retraining on the output of a

broken pipeline hides such a fault: the model is fitted to corrupted features, and breaks again when someone repairs the pipeline.

This paper asks an identifiability question: **from what evidence can a genuine change of the prediction problem be told apart from a defect of the data pipeline, and where is that impossible?** The answer has a clean half and a hard half, and the line between them is drawn by the data source. A changed component that the source did not force can be tested exactly by re-executing its known-good version, and monitoring cannot replace that test. When the source itself changed (a new layout, a recoded field, a new population), no re-execution can certify the new adapter, and any method that decides anyway needs an assumption about the world that real data can violate. A diagnoser should therefore say which half a component is in, decide where the data allow it, and hand the rest to a reviewer with the evidence it has. I make four contributions.

- **An identifiability theory (Section 4).** Silent defects are indistinguishable from world changes for every diagnostic that monitors model inputs and labels (Theorem 1). Re-execution decides unforced changes exactly: a rollback test detects local effectiveness and a propagation test end-to-end effectiveness (Theorem 2). It cannot certify source-forced changes (Theorem 3). From these I derive a three-verdict rule that never blames a healthy component and never misses a consequential defect silently, and show that a changed, consequential component it leaves unverified cannot in general be resolved without side information: any rule that resolves it risks one of the two errors (Proposition 1).
- **DoD-4 (Section 5),** a three-verdict diagnoser (DoD: *Drift or Defect*) that implements the rule. Three earlier versions, DoD to DoD-3, decide every case and use increasingly careful side information for forced changes: world smoothness, world-robust invariant evidence (Theorem 4), a rank-based history rule and validity conditions for re-execution evidence (Propositions 4–7); they test whether side information can close the gap that Theorem 3 opens.
- **A benchmark (Section 6)** of eleven executable pipelines over public data, including two three-table pipelines with joins of eleven and twelve components, with real, dated changes of the published files, real library-upgrade behaviours and replays of public incidents, in 2,961 evaluated cases whose ground truth comes from executing every action.
- **Four pre-registered rounds (Section 8, Section 9)** against the responses practitioners use, observational root-cause attribution, pipeline-instance debugging, bisection and, as supporting baselines, LLM agents with and without the same re-execution tools. I report negative results as found.

The evidence follows the boundary. Retrain-on-alarm and validation-first act only on alarms and failed checks, and accordingly diagnosed none of the 200 validation-silent consequential cases of rounds 1–3. On 267 of 267 consequential cases without a published source change in their period range, interventional diagnosis was correct, as Theorem 2 predicts when its assumptions hold, so these cases mainly confirm that they held in real pipelines (deterministic execution, faithful archives), while the baselines erred there as well. Where a change lay in the range, no method that commits to a diagnosis was reliable: the three deciding versions of DoD did not meet their pre-registered comparisons with the best baseline, and their revisions reduced false blame where their assumptions held but not the number of errors, as Theorem 3 predicts. The fourth round tested the alternative the theory implies: on two larger pipelines, DoD-4's false-blame rate was 0.000 and its silent-miss rate 0.000, and it decided 0.11 of the cases; the price of that

soundness is a review list of the components a published change touched. I report round 4 first and rounds 1–3 in summary; Appendix D gives their full reports.

2. Related work

Data validation and monitoring. TFDV [2], Deequ [5], Auto-Validate [6] and Auto-Validate-by-History [7] check schemas, ranges and learned constraints; PrismaDV [8] generates task-aware data tests with a language model. Drift detection [9], [10] and monitoring tools add distribution tests. These raise alarms but do not attribute them. Theorem 1 states precisely which defects any such check can see.

Attributing performance changes. DISDE [11], WhyShift [12] and hierarchical decompositions [13] attribute a performance drop to covariate and concept shift, assuming that features are computed correctly; a pipeline defect appears to them as one of these shifts. Budhathoki et al. [14] and Zhang et al. [15] attribute distribution or performance changes to changes in causal mechanisms; a pipeline step modelled as a mechanism could in principle be blamed, but from observational data alone, so Theorem 1 applies to them as well.

Debugging pipelines by re-execution. BugDoc [16] re-runs pipeline instances with known-good parameter values to find minimal failure causes; AID [17] and DataPrism [18] use causally guided interventions and minimise their number; mlinspect and mlwhatif [19], [20] instrument and patch pipelines; SAVAGE [21] searches for harmful corruptions. None models a world that may legitimately have changed, decides between fixing and retraining, or treats changes forced by a data source differently from free ones; my results show that this distinction is decisive.

Invariants. Daikon [22] infers likely program invariants from execution traces, and data validation systems learn constraints from data history [5], [7]. I use mined invariants differently: as evidence that is robust to arbitrary shifts of the world (Lemma 1), and I characterise the defects they cannot see (Theorem 4c). The rank argument behind my history rule is the one that underlies conformal prediction [23].

Causal diagnosis of ML systems. Paleyes and Lawrence [24] attribute anomalies on the dataflow graph observationally to input shift or faulty components; Leest et al. [25] propose causal system maps to separate software faults, data-quality issues and natural change, without an evaluation. Causal root-cause analysis for microservices [26], including interventional approaches [27], and root-cause discovery and identifiability results [28], [29] are the closest formal relatives; my results are their counterpart for data pipelines, where the confounder is the world itself. Observability for ML pipelines [30] and studies of data-pipeline issues [31], [32] motivate the problem.

Abstention. My three-verdict rule is a diagnostic counterpart of the reject option in classification [33] and of selective classification, which trades coverage for risk [34]. The difference is the source of the abstention: not a confidence score but an identifiability result: every component that Proposition 1(d) shows no test of the pipeline can resolve is among those left unverified.

3. Problem formulation

World records $W \in \Omega_W$ with labels $Y \in \Omega_Y$ follow a distribution D from a family $\mathscr{F}$ of distributions I am willing to entertain (covariate, label and concept shift). A *source encoding* σ, from a family $\mathscr{E}$ of encodings the provider might use, turns world records into published raw records $R = \sigma(W)$ (layout, units,

code lists); σ#D denotes the resulting law of raw records and labels. A pipeline is a directed acyclic graph of deterministic components; with versions θ (code, configuration, reference artifacts, library environment) it computes features X = P_θ(R), and f_c^θ denotes the function component c computes at version θ. θ^0 is the version deployed when the model h was trained; θ^* is the *specification* for the current period, that is, the pipeline a careful engineer would run on the current sources. A component c is *changed* if $\theta_c \neq \theta_c^0$ and *forced* if its specification differs from the reference ($\theta_c^* \neq \theta_c^0$, e.g. an adapter rewritten for a new layout). It is a *defect* if $f_c^{\theta_c}$ differs from $f_c^{\theta_c^*}$ on some input the specification pipeline can feed it, and *benign* if it is changed but not a defect (a refactor, a harmless upgrade, a correct adapter written differently from θ_c^*).

A defect c is *locally effective* if replacing θ_c by θ_c^*, with every other component at its current version, changes c's own output on its current inputs, and *effective* if this replacement changes the final pipeline output on the current records. Effectiveness is the end-to-end notion and defines the ground truth; a component is *healthy* in a case if it is not an effective defect. Because effectiveness is judged with the other components at their current versions, two defects can mask each other (a downstream defect that drops a column hides an upstream defect in that column), and fixing one can make the other effective. A raw-reading component (an *adapter*) maps published records to semantic records S = A(R); the correct adapter for period t satisfies $A_t^* \circ \sigma_t$ = s, where s : $\Omega_W \rightarrow \Omega_S$ is the semantic content the provider publishes. A published change lies *in the period range* of a case with reference period p_0 and current period p_1 if it was published in a period p with $p_0 < p \leq p_1$; the case then *spans* the change.

I distinguish three kinds of diagnostic by what they observe. A *monitoring-only* diagnostic sees samples of the joint law of model inputs and labels, (P_θ(σ(W)), Y), reference statistics, the lineage graph and the change log (which components changed and in which aspect). A *lineage-observational* diagnostic also sees the archived outputs of every component in both periods, and an *interventional* diagnostic may in addition re-execute components, at any version, on archived inputs. The actions are fix(S) (replace θ_c by θ_c^* for every c ∈ S), retrain, fix(S)+retrain and nothing; the benchmark scores a fix by executing the specification version, which for a forced component assumes that the engineer who acts on the diagnosis writes a correct adapter. I call an effective defect *consequential* if fixing it alone changes the held-out accuracy under the 'nothing' or under the 'retrain' action by at least 0.002 (held-out and retraining records are current records, and training is deterministic given its inputs); the correct action fixes the consequential defects and retrains iff retraining after the fix gains more than 0.01 accuracy. A diagnosis is *correct* if the set of components it would fix contains every consequential defect and only effective defects.

A *three-verdict* diagnosis in addition lists a set U of components it declares *unverified* (neither fixed nor cleared) for a human to review; it takes the same actions as a two-verdict diagnosis and leaves the components in U at their current versions until they are reviewed. It commits a *false blame* if it fixes a component that is not an effective defect, and a *silent miss* if a consequential defect is neither fixed nor in U. It *decides* a case if U is empty; it is *correct where decided* if it decides the case and is correct. Correctness as defined above counts the fix set only, so a three-verdict diagnosis that leaves a consequential defect in U is not correct. A two-verdict diagnosis is the special case U = ∅, for which correctness means no false blame and no silent miss.

4. Identifiability

I state the main results here and give their proofs in Appendix A; results on side information for forced changes are stated and proved in Appendix B. Theorems 1–3 (Theorem 2 without the propagation test) and Proposition 2 were stated before round 1, Lemma 1, Theorem 4 and Propositions 3–4 before round 2, Propositions 5–7 before round 3, and Theorem 2(b–c) and Proposition 1 before round 4 (Section 7).

4.1 What monitoring and re-execution can identify

Theorem 1 (observational equivalence of silent defects). Let (D, σ, θ) contain a defect. If some $D' \in \mathscr{F}$ and some θ' without defects with the same change-log signature (the same components changed in the same aspects) give the same joint law of model inputs and labels, then every monitoring-only diagnostic outputs "defect" with the same probability under both configurations; its type-I plus type-II error on the pair is at least 1.

Corollary 1 (image characterisation). If $\mathscr{F}$ contains all distributions on world records and labels, a defect is monitoring-equivalent to a world change if and only if its corrupted features lie almost surely in the image Img = $P_{\theta^*}(\sigma(\Omega_W))$ of the correct pipeline. A defect that puts mass $\varepsilon > 0$ outside Img is detected by a support check against Img with probability $1 - (1 - \varepsilon)^n$ from n records.

Validation of the model inputs can therefore catch at most the *loud* defects, and a support check against Img catches all of them; *silent* ones (inside the image) are invisible to drift tests, validation and label monitoring of the model inputs and to any learned combination of them, unless $\mathscr{F}$ is restricted or the diagnostic uses information beyond the joint law. Intermediate outputs are such information: the argument of Corollary 1 carries over to a lineage-observational diagnostic with the map from world records to the outputs of all components in place of $P_{\theta^*} \circ \sigma$, so a defect is invisible to it only if the joint law of all archived outputs is one the correct pipeline produces under some world distribution. Because a component's output is then seen together with its inputs, many silent defects fail this condition, which is why observational attribution finds some of them (Appendix D.2).

Theorem 2 (interventional identification of unforced changes). Let components be deterministic, the intermediate outputs of the production run be archived, the reference version of a changed component c and the current versions of the components downstream of it be re-executable, and let c not be forced ($\theta^*_c = \theta^0_c$). (a) The *rollback test* (run θ^0_c on c's current inputs and compare with c's current output) returns *differ* iff c is a locally effective defect. (b) Local effectiveness is necessary for effectiveness. (c) c is effective iff the *propagation test* (re-execute the components downstream of c, at their current versions, on the rolled-back output and compare the final output with the current one) returns *differ*; by (b) this can happen only when the rollback test differs. The tests need one execution of c and, if its rollback differs, one execution of its downstream subgraph. (d) If no component is forced ($\theta^* = \theta^0$), testing the changed components Δ in topological order, each on the inputs produced after the replacements upstream of it, and replacing those whose rollback differs yields the correct pipeline's output, so the remaining performance gap is attributable to the world (these sequential tests reconstruct P_{θ^*}(R); unlike (a)–(c), they do not classify components one by one).

Theorem 3 (the limit: source-forced changes). Let the encoding change from σ^0 to σ^1 at a published change, with the reference period fixed. (a) If some $D'' \in \mathscr{F}$ satisfies $\sigma^1\#\mathrm{D} = \sigma^0\#D''$, no diagnostic using monitoring, archived outputs and any re-execution of pipeline components separates (D, σ^1) from (D'', σ^0). (b) More generally, if some $\sigma^{1'} \in \mathscr{E}$ and $D' \in \mathscr{F}$ satisfy $\sigma^{1'}\#D' = \sigma^1\#\mathrm{D}$, no such diagnostic separates (D, σ^1) from (D', $\sigma^{1'}$); an adapter that is correct under one of them and not under the other is certified or rejected wrongly in one of the two. Part (a) covers changes whose new records the old encoding could have produced (a reinterpreted unit or code); part (b) also covers visible changes such as a renamed or new column or a new layout, whose meaning is a property of σ^1 that no re-execution observes unless $\mathscr{E}$ is restricted, for example by the provider's documentation of the change.

Side information. Theorem 3 leaves two options for a component that a published change touched: decide it with assumptions that restrict $\mathscr{F}$ or $\mathscr{E}$, or leave it undecided. Appendix B states the assumptions I studied: world smoothness at published changes, invariants of the provider's process that hold under any world shift (Lemma 1, Theorem 4, Proposition 3), a rank-based history rule (Proposition 4) and validity conditions for re-execution evidence (Propositions 5–7). It also gives a test-ordering result (Proposition 2). Appendix C describes the three versions of DoD that decide every case with them, and Section 9 reports how they fared. The next subsection gives the rule that decides only where the data allow it.

4.2 A sound three-verdict rule

Theorems 2 and 3 split the changed components of a case along a line that the change record and the column lineage can draw. Call a component *touched* if it reads, directly or through upstream components (by column-level lineage), a column that a published change in the case's period range names; a change of the set of published records (coverage, population) names the table's key column and so touches every component that reads the key. Whether a touched component is forced is itself a property of the new encoding (Theorem 3), so no rule may trust its rollback; an untouched component is unforced as long as the change record is accurate, so its rollback reproduces its specification.

Proposition 1 (a three-verdict rule). Assume deterministic components, archived intermediate outputs, a complete change log (every component with $\theta_c \neq \theta_c^0$ is listed), accurate column-level lineage and an *accurate change record*: every forced component is touched. Let T be the touched components and let the rule (i) list every component of T as unverified, (ii) fix a changed component outside T iff its rollback and propagation tests both return *differ*, and (iii) clear every other component; components whose reference version or propagation test cannot be run are listed as unverified, which overrides (ii) and (iii). Then (a) every fixed component is an effective defect, so the rule is *sound* (it never commits a false blame); (b) every consequential defect is fixed or unverified, so it is *complete* (it never commits a silent miss); (c) if T is empty and every test runs, U is empty and the rule fixes exactly the effective defects: it decides the case and its diagnosis is correct; and (d) let $c \in T$ be changed, let rolling it back change the held-out accuracy under the 'nothing' or under the 'retrain' action by at least 0.002, and let its deployed output be attainable by a correct adapter under some world distribution in $\mathscr{F}$ (Corollary 1). If $\mathscr{F}$ contains a world distribution that the new encoding maps onto the old (Theorem 3a), there are two configurations that agree on the raw-record law, the reference period, the change log and the change record, in one of which c is a correct adaptation and in the other a consequential defect; a rule using only monitoring, archived outputs and re-execution that fixes c commits a false blame in the first, and one that clears it a silent miss in the second.

Soundness alone is trivial (fixing nothing achieves it), and so is completeness (listing every component achieves it). The content of Proposition 1 is that one rule has both, is exact where nothing is touched (c), and cannot shorten its review list for changed, consequential touched components without side information (d). The assumption on T is one-sided: any T that contains every forced component will do, so a component known to be unforced (such as the step that applies the model's fitted encoder, whose specification is its reference version) may be left out of T. For the components covered by (d), the unverified verdict is the price of soundness, not a weakness of a particular test: without assumptions that restrict $\mathscr{F}$ or $\mathscr{E}$, evidence about such a component can only rank it for review, and assumptions such as (I) can establish that it is a defect (Theorem 4a, Proposition 7) but never clear it (Theorem 4c).

5. DoD-4: the three-verdict diagnoser

DoD (*Drift or Defect*) receives the monitoring report of the reference and current periods, archived intermediate outputs of both production runs and of the deployment history, the lineage graph with column-level lineage to raw columns, the change log and the dated published changes of each source. I built it in four versions, each frozen before it was evaluated. The first three (DoD, DoD-2, DoD-3; Appendix C) decide every case and differ only in how they judge components that a published change touched, using the side information of Appendix B. DoD-4, described here, does not judge them.

DoD-4 implements Proposition 1; its fix, clear and review verdicts use no thresholds:

- **Touched components.** A component is touched if, by column-level lineage, it reads directly or through upstream components a column that a published change in the case's period range names. The pipeline's output step, which applies the model's fitted encoder, is never touched: the encoder is versioned with the model and changes only by retraining, so its specification is its reference version and Proposition 1 allows leaving it out of T. A change of a table's record set names the table's key column.
- **Verified defects.** Every changed component that is not touched gets the rollback test; if it differs, the pipeline is re-executed with that component rolled back and every other at its current version, and the final outputs are compared (propagation test, Theorem 2c). Components for which both differ are fixed; the others are cleared as healthy.
- **Unverified components.** Every touched component, changed or not, and every component whose reference version or propagation test cannot be run, is listed for review and never fixed. DoD-4 labels each with DoD-3's evidence (*suspect* if DoD-3 would fix it, *no evidence* otherwise) as a ranking for the reviewer; *DoD-4 lite* omits the labels and has the same fix and review sets.
- **Action.** Fix the verified defects and leave the unverified components at their current versions; retrain by DoD-3's rule (DoD-4 lite, which does not run DoD-3: if the accuracy drop is at least 0.01).

6. Benchmark

The benchmark consists of eleven executable pipelines over public data, added in four rounds; each round's domains were built before any method was run on them (Table 1). The pipelines of rounds 1–3 have four to seven components: a source adapter, feature and cleaning steps (some deployed in a pandas 1.5 environment), reference-data joins, and categorical encoding with a fitted artifact. The two round-4

pipelines read three source tables each (storm events with their fatality and location records, traffic crashes with the people and vehicles involved) and have twelve and eleven components: three adapters, a branch of time and code-mapping steps (time-zone table, tornado-scale map and source crosswalk; notification delay and trafficway crosswalk), per-record aggregates of the two child tables with their own code crosswalks, a three-input join, a spatial grid, cleaning and encoding. Every domain has real, dated changes of the published files (found by profiling the raw files of every period, not from documentation alone), except the airline domain, whose history is republished in the current layout and which serves as a control.

Table 1. Benchmark domains. Records: sampled raw rows used by the pipelines, summed over all source tables. Bike share: 100,000 trips per quarter, plus the 3,638-row station table for Chicago bike share; census: person and household records of a 30% sample of households in California, New York and Texas; 150,000 records per period for mortgages, airlines and house sales; for taxi, a uniform random sample of about 150,000 trips per month, so its periods hold between 149,342 and 150,910 trips; for Chicago crashes, 60,000 crashes per year with their person and vehicle records (all 44,297 crashes of 2016, when fewer were published); all records for news, fatal crashes and storms. Periods: airlines use January, April, July and October of each year (47 months); taxi uses the same months plus June 2016, December 2024 and February 2025 (50 months). Changes: periods with a dated published change from the first reference period on. In total 40,932,905 records and 3,382 cases, of which 2,961 were evaluated (storm: a fixed subsample, Section 7).

Round	Source	Periods	Records	Steps	Task	Changes	Cases
1	US Census ACS 1-year PUMS [35][1]	2014–2024 (11)	3,995,334	7	income > $50k	4	274
1	NYC TLC yellow-taxi trips [36]	2015-01–2026-07 (50)	7,502,536	6	tip ≥ 20% of fare	6	326
1	HuffPost News Category [37]	2012–2022 (11)	209,527	4	27 sections	4	102
2	Divvy trip data [38]	2014-Q1–2026-Q3 (51)	5,103,638	5	member vs casual	10	396
2	HMDA loan records [39]	2012–2025 (14)	2,100,000	4	denied vs originated	6	280
2	BTS on-time performance [40]	2015-01–2026-07 (47)	7,050,000	5	arrival delay ≥ 15 min	0	324
3	Capital Bikeshare trip data [41]	2014-Q1–2026-Q3 (51)	5,100,000	5	member vs casual	5	312
3	NHTSA FARS crash file [42]	2010–2024 (15)	509,887	4	two or more vehicles	6	240
3	HM Land Registry Price Paid [43]	2008–2025 (18)	2,700,000	5	new build	1	257
4	NOAA Storm Events Database [44]	1996–2025 (30)	3,604,343	12	property damage	8	640
4	Chicago Traffic Crashes [45]	2016–2025 (10)	3,057,640	11	injury or tow	3	231

Real source changes. Appendix E lists every change (Table 11). They include renamed and reordered columns, new layouts after an operator's system change (both bike-share systems in the first half of 2020, as the pandemic began), recoded categories (member/casual; loan purpose and occupancy under amended Regulation C in 2018), changed units (mortgage amounts in dollars instead of thousands), removed fields (crash weather fields in 2020), a byte-order mark that appeared in a header for two years, timestamp

[1] The Census Bureau did not release its standard 2020 ACS 1-year data, because the COVID-19 pandemic disrupted data collection; it released experimental 1-year estimates and a PUMS file with experimental weights instead. The 2020 period of the census domain uses that experimental file, which in my sample has about a fifth fewer households than the 2019 file.

formats with and without seconds or fractions, coverage changes (non-standard house sales published from 2013; storm locations only for events with coordinates from 2006; citywide crash reporting in Chicago from 2017), code lists that were extended, renamed or recased (storm time zones with the UTC offset appended in 2006, reporting sources republished in title case, tornado ratings on the Enhanced Fujita scale from 2007; crash restraint and vehicle contact-point codes in 2019 and 2020). Several coincide with genuine world changes: COVID-19 (2020), an airport-fee rise in the quarter the taxi fee column was renamed (April 2023) and congestion pricing with its new column (January 2025).

Library behaviours. I created pandas 1.5.3, 2.2.3 and 3.0.6 environments and probed idioms common in pipeline code; several behave differently, including chained assignment that becomes a no-op under copy-on-write [46] (pandas warns [47]; pipelines that suppress warnings fail silently) and timestamps whose integer representation changes resolution [46], [48]. Components declare their environment and run in it as subprocesses, so a library upgrade is a real execution, not a simulation.

Defects and decoys. Each domain has a catalog of changes in five tiers (Appendix E, Table 12): T1 defects tied to a published change (a stale mapping, a format not updated, a positional column mapping; cf. a renamed census field that broke a public benchmark library [49]); T2 real library-upgrade behaviours; T3 documented incident classes (unit errors, stale reference data, copy-paste join keys, swapped day and month, encoders re-fitted at serving time); T3r replays of public incidents [50]–[52]; and benign changes (refactors, harmless upgrades, correct adapters). A case pairs a reference period with a current period and applies nothing, benign changes only, one defect with up to two benign decoys, or two defects, on top of the correct pipeline of the current period. A changed component is therefore not by itself a defect: every domain has cases with benign changes only, and most defect cases also contain a decoy.

Ground truth by execution. Records are split by hashed id into training (40%), serving (30%, what diagnosers see) and held-out (30%, grader only, with labels from the correct pipeline). For every case I execute every canonical action and every action a method recommends, and score it on the held-out records.

7. Evaluation protocol: four pre-registered rounds

I evaluated in four rounds (Table 2). In each, hypotheses, endpoints, thresholds and a SHA-256 digest of all code and configuration were frozen before any test case was run, and the case runner refused test cases if the digest differed; the digests and frozen files are part of the released repository (Appendix G). After each round, the next version of the method was designed on data already seen (the earlier rounds' domains, from then on development data) and tested on fresh domains. In round 1, the census domain was used to design the method, and the taxi and news cases were split into development and test cases by defect type (cases without a defect by period: every third current period is development); only test cases count. From round 2 on, every case of a new domain is a test case.

Table 2. The four pre-registered rounds. Every method of earlier rounds is run unchanged in later rounds.

Round	Test domains	New method	Hypotheses	Digest	Files
1	Census (development), taxi, news	DoD	H1–H5	8eab1e0e42e9…	30
2	Chicago bike share, mortgages, airlines	DoD-2	H6–H10	e50ee051bbc5…	25
3	Washington bike share, fatal crashes (FARS), house sales	DoD-3	H11–H14	fb3229bdedc2…	33

Round	Test domains	New method	Hypotheses	Digest	Files
4	Storm events, Chicago crashes	DoD-4	H15–H19	9543f4c5d4e8…	36

For compute reasons, the storm domain was evaluated on a fixed random subsample of 219 of its 640 cases (one in three, selected by a hash of the case identifier); all other domains were evaluated in full.

Baselines. *Retrain on alarm* retrains if accuracy on labelled serving data dropped by at least 0.01. *Validation first* learns missing-value and range checks on reference data, routes a failure through lineage to the changed or source-reading component closest to the failing input, and otherwise retrains on alarm. *Observational attribution* blames the changed or source-coupled component that introduces the most drift on the dataflow graph, using archived outputs of both production runs (in the spirit of [24]). *Full rollback*, *bisection* and *pipeline-instance debugging* (BugDoc's shortcut [16]) re-run whole pipelines with known-good versions swapped in. The *LLM agent* receives the same report, lineage, change log and published changes as all methods; in the *tools* variant it may also call the rollback, replay (a whole-pipeline run at chosen versions), effect and boundary tests, up to eight calls within a compute budget of three full pipeline runs. LLM agents are supporting baselines: they show what a general-purpose model makes of the same evidence, not an upper bound. I used OpenAI's gpt-5.6-luna on all cases (round 4: every chicago case and every second storm case) and gpt-5.6-terra on test cases with a consequential defect (round 1: all of them; rounds 2 and 3: every third, round 4: every fourth, within each round's API allowance; reasoning effort low, seed 0, responses cached for replay).

Metrics. In rounds 1–3 the primary endpoint is diagnosis correctness on cases with a consequential defect; secondary metrics are false blame (a blamed component is not an effective defect), missed defects, regret (held-out accuracy of the best action minus that of the recommended one), fragility (the accuracy a retrained model loses once the pipeline is repaired) and measured diagnosis compute. Round 4 makes the two errors of a diagnosis primary: false blame over all cases and silent misses over cases with a consequential defect, together with coverage, correctness where the method decides, and review load (Section 3). Confidence intervals are 95% cluster-bootstrap intervals (2,000 resamples; clusters are domain × defect signature); tests within a round are Holm-corrected. Rounds are reported side by side and never pooled for a test.

8. Round 4: two multi-table pipelines (DoD-4)

Table 3. Round 4 (all cases are test cases). False blame over all cases; silent misses and two-verdict correctness over cases with a consequential defect; coverage is the share of cases a method decides (lists no component for review); review load is the mean number of components listed for review. Methods other than DoD-4 and DoD-4 lite always decide. LLM agents ran within the round's pre-registered API allowance: luna on 330 of the 341 planned cases (every Chicago case, every second storm case), terra on 18 cases of its sample of every fourth consequential case; their rates are over the cases they ran on.

Method	False blame (n=450)	Silent miss (n=256)	Coverage	Correct where decided	Correct (n=256)	Review load
Retrain on alarm	0.00	1.00	1.00	0.43	0.00	0.00
Validation first	0.83	0.59	1.00	0.13	0.08	0.00
Observational attribution	0.82	0.80	1.00	0.16	0.20	0.00
Full rollback	0.28	0.47	1.00	0.46	0.05	0.00
Bisection	0.02	0.48	1.00	0.71	0.49	0.00
Pipeline-instance debugging	0.03	0.39	1.00	0.75	0.56	0.00

Method	False blame (n=450)	Silent miss (n=256)	Coverage	Correct where decided	Correct (n=256)	Review load
LLM agent, report (luna)	0.88	0.44	1.00	0.12	0.12	0.00
LLM agent, tools (luna)	0.32	0.36	1.00	0.57	0.48	0.00
LLM agent, report (terra)	–	0.22	–	–	0.28	–
LLM agent, tools (terra)	–	0.33	–	–	0.44	–
DoD	0.04	0.27	1.00	0.81	0.68	0.00
DoD-2	0.04	0.27	1.00	0.82	0.68	0.00
DoD-3	0.04	0.32	1.00	0.79	0.64	0.00
DoD-4 lite	**0.00**	**0.00**	**0.11**	**1.00**	**0.61**	**5.28**
DoD-4	**0.00**	**0.00**	**0.11**	**1.00**	**0.61**	**5.28**

H15 (soundness): supported. DoD-4's false-blame rate (the share of cases in which it fixed a component that was not an effective defect) was 0.000 over the 450 cases (threshold 0.02: met); on the cases spanning a published change, its false-blame rate was lower than DoD-3's (paired difference -0.05, 95% CI [-0.10, -0.01], Holm $p < 0.001$). DoD-4's fixes are verified by two exact re-executions, so a false blame could only come from a failure of Proposition 1's assumptions: a component that a published change forced but whose columns the change record does not name, or non-deterministic execution.

H16 (completeness): supported. A consequential defect was neither fixed nor listed for review in a share of 0.000 of the 256 cases with one (threshold 0.02). **H17 (exact where decided): supported.** DoD-4 decided 0.11 of all cases; its diagnosis was correct on 1.00 of the decided cases (n = 51) and on 1.00 of the decided cases with a consequential defect (n = 30).

H18 (identifiable regime): supported. On the 30 consequential cases without a published change in the period range, DoD-4 was correct in 1.00; the best non-DoD methods run on all of them, pipeline-instance debugging and bisection, in 0.83.

H19 (evidence for review, descriptive). DoD-4 listed 5.28 components per case for review (0.46 of a pipeline's components). Of the 65 labelled *suspect* by DoD-3's evidence, 46 were effective defects; of the 2312 labelled *no evidence*, 154 were (AUC of the label for effective defects 0.61).

By domain, storm events had coverage 0.12, review load 6.53 and false blame 0.000 (DoD-3 0.06); Chicago crashes had coverage 0.11, review load 4.10 and false blame 0.000 (DoD-3 0.03).

Soundness and completeness have a price, which the two-verdict endpoint shows. Counting only what a method fixes, DoD-4 was correct on 0.61 of the consequential cases, against 0.68 for DoD-2, the best method that always commits: where a consequential defect sat in a component a published change touched, DoD-4 listed it for review instead of fixing it, and a method that commits fixed it whenever its side information happened to point the right way, at the cost of false blames (0.038 of all cases) and silent misses (0.270 of consequential cases) that DoD-4 did not make. The review list is long (0.46 of a pipeline's components), and DoD-3's evidence ranks it only partly: the *suspect* label is precise but covers 0.23 of the effective defects on the list.

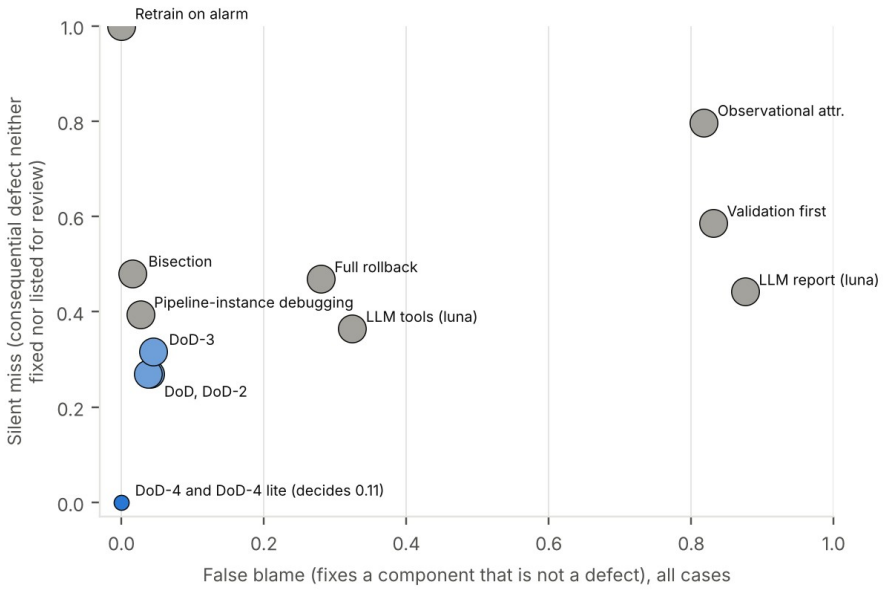


Fig. 1. Round 4: false blame (all cases) against silent misses (cases with a consequential defect) for every method run on most cases (LLM agents: the cases within the API allowance, Table 3). Methods that always commit to a diagnosis trade one error for the other; DoD-4 avoids both by listing the components a published change touched for review (marker size: share of cases decided).

9. Rounds 1–3: can side information decide forced changes?

Rounds 1–3 tested the three deciding versions of DoD, each on three domains it had not seen, against the same baselines; Table 4 summarises their pre-registered primary comparisons, and Appendix D reports every hypothesis, the results per domain and the analysis of each round's failures.

Table 4. Pre-registered primary comparisons of rounds 1–3: correct diagnosis on cases with a consequential defect for the round's DoD version and for the best baseline (round 1: test split). Each rate is over the cases the method was run on; the paired difference is over the cases both were run on (for the sampled terra agents in rounds 2 and 3, the n given), so it need not equal the difference of the two rates. The verdicts apply the pre-registered criteria (Appendix F): in rounds 1 and 2 a difference of at least 0.15 with the 95% interval excluding 0 (round 1's +0.14 is significant but below that minimum), and in round 3 a Holm-adjusted $p < 0.05$ (minimum effect of interest 0.10).

Round	Test domains	Method	Cons. cases	Correct	Best baseline	Paired difference [95% CI], Holm p	Verdict
1	Census (dev.), taxi, news	DoD	97	0.74	LLM agent, tools (terra) 0.60	+0.14 [+0.05, +0.25], p = 0.008	not supported
2	Chicago bike share, mortgages, airlines	DoD-2	435	0.40	LLM agent, tools (terra) 0.80	-0.16 [-0.37, +0.02] (n = 49), p = 0.368	not supported
3	Washington bike share, FARS, house sales	DoD-3	373	0.46	LLM agent, tools (terra) 0.56	-0.05 [-0.33, +0.26] (n = 41), p = 0.882	not supported

None of the deciding versions met its comparison with the best baseline, and the side information helped only where its assumptions held. DoD-3 blamed fewer healthy components than DoD on cases spanning a published change (difference -0.05, Holm p = 0.009) and made the same diagnosis as DoD on 1.00 of the cases without one, but it did not diagnose more consequential defects correctly (difference +0.02, Holm p = 0.882). The split that Theorems 2 and 3 predict was sharp: without a published change in the period range, every version was correct on every consequential case (Section 10); with one, no version exceeded

0.68. Monitoring alone behaved as Theorem 1 predicts: retrain-on-alarm and validation-first diagnosed none of the 200 validation-silent consequential cases, while observational attribution, which also reads intermediate outputs, found some.

Every remaining failure was a forced change whose correctness depends on what the new encoding means, which is the situation of Theorem 3. The world jumped in the same quarter as a published change (the taxi airport fee rose when its column was renamed in April 2023; bike-share trip durations jumped at the 2020 layout changes), a change of the published population was intended (newly published non-standard house sales from 2013, which the correct pipeline drops), and values were missing by design (dockless bike-share trips have no station). Each assumption the deciding versions added (that the world does not jump at a change, that records are not removed, that a provider-defined field is always defined) was false somewhere. DoD-4 therefore decides only where Theorem 2 applies.

10. Across the four rounds

This section collects analyses that were not pre-registered; they use the frozen records of all four rounds and are reported per round.

10.1 The identifiability boundary in the data

Table 5. Correct diagnosis on consequential cases, split by whether a published source change lies in the case's period range (a case-level proxy for the boundary of Proposition 1, which separates components). Best other method: the non-DoD method with the highest rate among those run on all of the cases. Round 1: test split.

Round	Change in range	n	Round's DoD version	Correct	DoD (round 1)	Best other method	Correct
1	no	19	DoD	1.00	1.00	Bisection / Pipeline-instance debugging / LLM agent, tools (luna) / LLM agent, tools (terra)	0.79
1	yes	78	DoD	0.68	0.68	LLM agent, tools (terra)	0.55
2	no	121	DoD-2	1.00	1.00	LLM agent, tools (luna)	0.73
2	yes	314	DoD-2	0.16	0.10	Observational attribution	0.31
3	no	97	DoD-3	1.00	1.00	LLM agent, tools (luna)	0.78
3	yes	276	DoD-3	0.27	0.25	Observational attribution	0.43
4	no	30	DoD-4	1.00	1.00	Bisection / Pipeline-instance debugging	0.83
4	yes	226	DoD-4	0.55	0.64	Pipeline-instance debugging	0.52

Without a published change, the round's DoD version was correct on 267 of 267 consequential cases over the four rounds (Table 5); this is the regime in which Proposition 1(c) makes re-execution exact. With a change in the period range, no method run on all cases exceeded 0.68 in rounds 1–3, and no version of the side information I tried (smoothness, invariants, history rules, introduced missingness) closed the gap (Fig. 2). In round 4 the best method that commits to a diagnosis reached 0.64 (DoD-2) on the 226 consequential cases with a change in range, and DoD-4 0.55 with its fix set alone; it fixed nothing it had not verified. In 125 of these 226 cases no consequential defect was in a component the change touched, and DoD-4 was correct on 125; on the rest it listed the consequential defects for review.

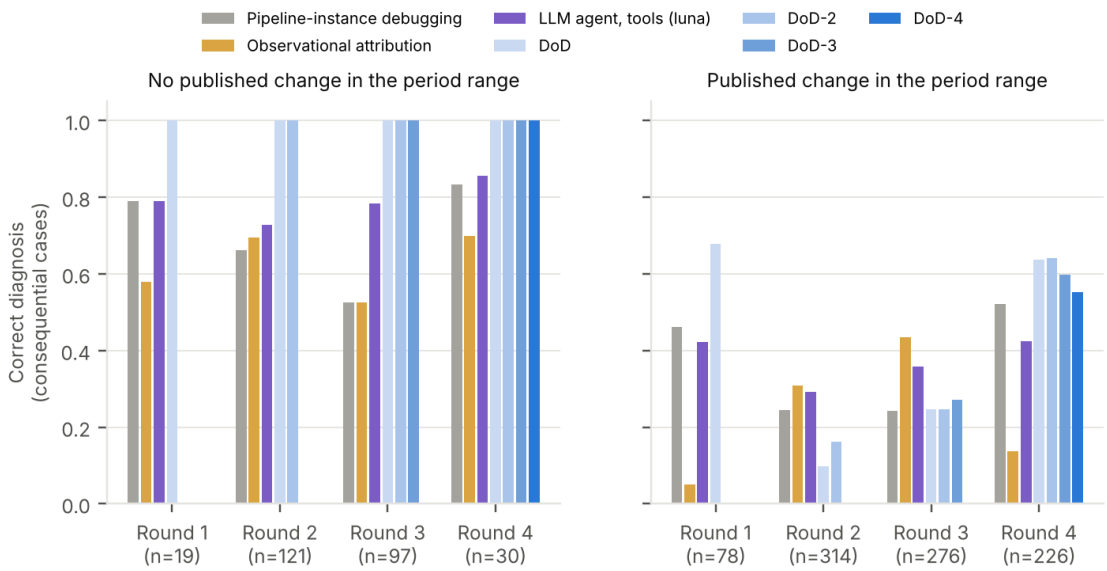


Fig. 2. Correct diagnosis on consequential cases without (left) and with (right) a published change in the period range, per round. DoD-4 (round 4) lists the components a change touched for review instead of fixing them, so on the right it is correct only where the consequential defects were untouched.

10.2 The two errors of a diagnosis

Table 6. False blame (share of all cases in which a fixed component is not an effective defect) / missed consequential defects (share of consequential cases in which one is neither fixed nor, for DoD-4, listed for review), per round. Round 1: test split. Methods are listed where they were run on all cases of a round; in round 4 the luna agents ran on the cases within the round's API allowance (Table 3).

Method	Round 1	Round 2	Round 3	Round 4
Retrain on alarm	0.00 / 1.00	0.00 / 1.00	0.00 / 1.00	0.00 / 1.00
Validation first	0.48 / 0.73	0.73 / 0.57	0.39 / 0.63	0.83 / 0.59
Observational attribution	0.67 / 0.85	0.55 / 0.58	0.49 / 0.54	0.82 / 0.80
Full rollback	0.30 / 0.32	0.07 / 0.77	0.13 / 0.73	0.28 / 0.47
Bisection	0.14 / 0.34	0.00 / 0.77	0.05 / 0.74	0.02 / 0.48
Pipeline-instance debugging	0.11 / 0.33	0.07 / 0.62	0.11 / 0.64	0.03 / 0.39
LLM agent, report (luna)	0.56 / 0.64	0.64 / 0.52	0.59 / 0.39	0.88 / 0.44
LLM agent, tools (luna)	0.32 / 0.37	0.41 / 0.51	0.30 / 0.44	0.32 / 0.36
DoD	0.33 / 0.08	0.54 / 0.07	0.47 / 0.18	0.04 / 0.27
DoD-2	–	0.52 / 0.07	0.47 / 0.18	0.04 / 0.27
DoD-3	–	–	0.44 / 0.20	0.04 / 0.32
DoD-4	–	–	–	0.00 / 0.00

Every method that commits to a diagnosis traded one error for the other or made both (Table 6). Retrain on alarm never blamed a component and missed every consequential defect; bisection and pipeline-instance debugging rarely blamed a healthy component but missed 0.33 to 0.77 of the consequential defects, depending on the round. In rounds 1–3 the DoD versions missed fewer but blamed a healthy component in 0.33 to 0.54 of all cases, and 1.00 of those false blames (rounds 1–3, all versions) fell in cases spanning a published change, where Theorem 3 limits what re-execution can decide; observational attribution, validation first and the LLM agent reading the report did badly on both. DoD-4 is the only method in the study with both rates near zero; its cost is the review list.

10.3 Local versus end-to-end effectiveness

Theorem 2 separates the two notions. Over the 3,577 rollback tests that the DoD versions ran on components that were not forced ($\theta_c^* = \theta_c^0$; each case and component counted once), a *differ* verdict marked an effective defect in 1,575 tests and a locally effective but not effective one in 1 (a timestamp-resolution defect from a library upgrade, in a case with a second, consequential defect upstream; its difference did not reach the final output), and an *equal* verdict was never followed by an effective defect (0 of 2,001), as Theorem 2(b) requires. On forced components the same test was uninformative: of 2,374 *differ* verdicts only 356 concerned an effective defect (the reference version usually cannot read the new encoding at all), and 12 *equal* verdicts hid one.

In round 4, DoD-4 ran 218 propagation tests, on unforced components whose rollback differed; every one returned *differ*, and every one concerned an effective defect, so none of the tested components was masked downstream.

10.4 DoD-4 on the development domains

After the freeze, I ran DoD-4 on all cases of the nine round-1/2/3 domains (development data from round 4 on; 2,511 cases; its rule was designed on these domains, so the results are in-sample). Its false-blame rate was 0.000 and its silent-miss rate 0.000; it decided 0.30 of the cases, was correct on 1.00 of those, and listed 2.31 components per case for review (0.45 of a pipeline's components, about the same share as in round 4). In the development run of DoD-4, DoD-3 had been run only on the 809 cases of the three round-3 domains, so I compare the two methods on those 809 cases alone. There DoD-4's false-blame rate was 0.000 against 0.44 for DoD-3, and its silent-miss rate was 0.000 against 0.20 on the 373 consequential cases. Counting only what each method fixes, DoD-4 was correct on 0.51 of those consequential cases and DoD-3 on 0.46; DoD-4 decided 0.30 of the 809 cases, was correct on 1.00 of those, and listed 1.78 components per case for review (0.37 of a pipeline's components).

10.5 Sensitivity to single domains and single changes

Leaving out one domain or all cases that span one published change moves each round's primary endpoint considerably: round 1 (DoD: all 0.74, leaving out one domain 0.48–1.00, one published change 0.56–0.97); round 2 (DoD-2: all 0.40, leaving out one domain 0.21–0.66, one published change 0.41–0.60); round 3 (DoD-3: all 0.46, leaving out one domain 0.35–0.75, one published change 0.40–0.77); round 4 (DoD-4: all 0.61, leaving out one domain 0.41–0.86, one published change 0.48–0.80). Round-level conclusions about which method diagnoses more consequential defects therefore rest on few domains and a handful of changes, which is why I keep rounds separate and never pool them for a test.

DoD-4's error rates, unlike its correctness, do not depend on the composition: under every leave-one-out set its false-blame rate was 0.000 and its silent-miss rate 0.000.

11. Discussion and threats to validity

What the results say. Monitoring alone cannot separate silent defects from drift; re-executing components at their known-good versions can, exactly and with one execution per changed component (and one of its downstream subgraph per difference), for every change the data source did not force, provided the change record says correctly which changes those are. That half of the problem is solved by

Theorem 2 and by the engineering it requires: archived intermediate outputs, reproducible component versions including library environments, and column-level lineage. The other half, whether a component rewritten for a new source encoding is correct, is not, and three rounds of a method designed for it show why. Each piece of side information I used was a statement about the world or the provider that real data violated somewhere: the world does not jump at a published change (taxi fees in 2023, the pandemic quarter of the bike-share layout change); records are not removed at a change (house sales, where the correct pipeline must drop a newly published population); a field the provider defines is always defined (dockless trips, which have no station). Invariants that hold for every world shift (Lemma 1) are the principled alternative, but the four templates I mined almost never covered the defects of my catalogs (Theorem 4c). Proposition 1 turns this negative result into a design rule: decide exactly where the data allow it, and say so where they do not.

The two errors. Both are costly, in different ways. A diagnoser that blames a healthy component sends an engineer to rewrite code that works and, if the fix is applied, can break a correct adaptation; one that misses a defect leaves the pipeline broken and invites a retraining that hides it. Methods that always commit trade one error for the other (Fig. 1): retrain-on-alarm never blames a component and misses every defect, observational attribution and the first three DoD versions blamed healthy components at most published changes in some domains. A three-verdict diagnosis avoids both at the cost of a review list, and its value depends on how short and how well ranked that list is.

What a deployment should do. (1) Archive the intermediate outputs of production runs and keep the reference version of every component runnable, including its library environment; without them neither test of Theorem 2 can be run. (2) Keep a machine-readable record of the source's published changes (I built these records by profiling the raw files of every period) and column-level lineage; together they define which components a change touched, and Proposition 1's guarantees are only as good as that record. (3) Fix only components whose rollback and propagation tests both differ. (4) Treat a component that a published change touched as unverified, rank such components by the available evidence, and obtain the missing evidence from the provider (an overlap period in which both encodings are published, a change note that says which quantities changed) or from a review of the new adapter. (5) Consider deferring retraining while an adapter that feeds the model is unverified, since retraining fits the model to a possible defect; DoD-4 as evaluated retrains by DoD-3's rule, and my metrics do not score this choice.

Threats to validity. (i) The source changes and library behaviours are real, but most faulty versions are mine, built from documented incident classes. (ii) The pipelines have four to twelve components; the review load of Proposition 1 grows with the number of components a change touches. (iii) I wrote the change records and the column lineage from profiles of the published files; Proposition 1's guarantees hold only if they are accurate. (iv) Each version was designed on the earlier rounds' data, so development results are in-sample. (v) The consequential threshold, the retrain rule and the held-out split, fixed before round 1, shape the ground truth. (vi) The LLM agents used one prompt and low reasoning effort, and the larger model ran on samples. (vii) The split by published changes (Section 10) is post hoc, and round-level results are sensitive to single domains.

12. Conclusion

Telling drift from a broken pipeline is not a matter of better drift statistics. For silent defects it is impossible from monitoring data; for every change the data source did not force, re-executing the known-

good version settles it exactly; and for changes the source forced, no re-execution can. In four pre-registered rounds the evidence followed this boundary: interventional diagnosis was exact where no published change intervened, and no method that commits to a diagnosis was reliable where one did. A diagnoser that respects the boundary, fixing only what it has verified and listing what it cannot verify, avoided both false blame and silent misses on two larger pipelines. Better methods for the remaining half will need evidence from the provider rather than from the pipeline. I release the benchmark, code and records so that they can be compared on exactly these cases.

Declarations

Funding: This research was self-funded by the author and received no external funding.

Competing interests: The author declares no competing interests.

Author contributions: Ankitha Vuyyala: conceptualization, methodology, software, investigation, validation, formal analysis, visualization, and writing (original draft, review and editing). The author reviewed and approved the final manuscript and accepts responsibility for its content.

Data and code availability: The DoD implementation (all four versions), the baselines, the eleven executable pipelines with their defect catalogs and records of published source changes, the frozen study protocols and analysis plans, the raw case records of all four rounds and of the development runs, and the analysis and paper-build scripts are publicly available at https://github.com/ankitha-vuyyala/drift-or-defect. The artifacts corresponding to this manuscript are archived in release v1.0 at https://github.com/ankitha-vuyyala/drift-or-defect/releases/tag/v1.0. The repository includes the response cache of the 17,078 paid model calls, whose token usage and cost are recorded per model in runs/cost_ledger.json (Table 14). The case records regenerate every reported number, table and figure (paper/evidence_index.md maps each number of the build-script version of the paper to the file it was computed from), and the response cache replays every model decision without an API key. The source data are public and are not redistributed; the download scripts fetch them from their providers and check them against SHA-256 manifests: ACS PUMS (U.S. Census Bureau), NYC TLC trip records, the HuffPost News Category dataset, Divvy and Capital Bikeshare trip data, HMDA records (CFPB and FFIEC), BTS on-time performance data, NHTSA FARS, NOAA NCEI Storm Events, City of Chicago Traffic Crashes, and HM Land Registry Price Paid Data (contains HM Land Registry data © Crown copyright and database right, licensed under the Open Government Licence v3.0). Third-party data remain subject to their providers' licenses and attribution requirements.

Corresponding author: Ankitha Vuyyala. Email: ankithavuyyala08@gmail.com

References

[1] D. Papasian and T. Underwood, "How ML breaks: A decade of outages for one large ML pipeline," presented at USENIX Conf. Operational Machine Learning (OpML), 2020.

[2] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in Proc. Conf. Machine Learning and Systems (MLSys/SysML), 2019.

[3] S. Shankar, R. Garcia, J. M. Hellerstein, and A. G. Parameswaran, "'We have no idea how models will behave in production until production': How engineers operationalize machine learning," Proc. ACM Hum.-Comput. Interact., vol. 8, no. CSCW1, 2024.

[4] A. Shukla, K. Nath, N. Vats, V. Tatipathri, and D. Jagannathan, “D3: An automated system to detect data drifts,” Uber Engineering Blog, Feb. 23, 2023. [Online]. Available: https://www.uber.com/blog/d3-an-automated-system-to-detect-data-drifts/

[5] S. Schelter, D. Lange, P. Schmidt, M. Celikel, F. Biessmann, and A. Grafberger, “Automating large-scale data quality verification,” Proc. VLDB Endow., vol. 11, no. 12, pp. 1781–1794, 2018.

[6] J. Song and Y. He, “Auto-Validate: Unsupervised data validation using data-domain patterns inferred from data lakes,” in Proc. ACM SIGMOD Int. Conf. Management of Data, 2021, pp. 1678–1691.

[7] D. Tu, Y. He, W. Cui, S. Ge, H. Zhang, S. Han, D. Zhang, and S. Chaudhuri, “Auto-Validate by-History: Auto-program data quality constraints to validate recurring data pipelines,” in Proc. ACM SIGKDD Conf. Knowledge Discovery and Data Mining, 2023, pp. 4991–5003.

[8] H. Chen, A. Phani, and S. Schelter, “PrismaDV: Automated task-aware data unit test generation,” arXiv:2604.21765, 2026.

[9] S. Rabanser, S. Günnemann, and Z. C. Lipton, “Failing loudly: An empirical study of methods for detecting dataset shift,” in Advances in Neural Information Processing Systems (NeurIPS), 2019.

[10] Z. C. Lipton, Y.-X. Wang, and A. J. Smola, “Detecting and correcting for label shift with black box predictors,” in Proc. Int. Conf. Machine Learning (ICML), 2018.

[11] T. T. Cai, H. Namkoong, and S. Yadlowsky, “Diagnosing model performance under distribution shift,” Operations Research, vol. 74, no. 2, pp. 898–916, 2026, doi: 10.1287/opre.2023.0217.

[12] J. Liu, T. Wang, P. Cui, and H. Namkoong, “On the need for a language describing distribution shifts: Illustrations on tabular datasets,” in Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2023.

[13] H. Singh, F. Xia, A. Subbaswamy, A. Gossmann, and J. Feng, “A hierarchical decomposition for explaining ML performance discrepancies,” in Advances in Neural Information Processing Systems (NeurIPS), 2024.

[14] K. Budhathoki, D. Janzing, P. Blöbaum, and H. Ng, “Why did the distribution change?,” in Proc. Int. Conf. Artificial Intelligence and Statistics (AISTATS), PMLR vol. 130, 2021, pp. 1666–1674.

[15] H. Zhang, H. Singh, M. Ghassemi, and S. Joshi, “‘Why did the model fail?’: Attributing model performance changes to distribution shifts,” in Proc. Int. Conf. Machine Learning (ICML), PMLR vol. 202, 2023, pp. 41550–41578.

[16] R. Lourenço, J. Freire, and D. Shasha, “BugDoc: Algorithms to debug computational processes,” in Proc. ACM SIGMOD Int. Conf. Management of Data, 2020.

[17] A. Fariha, S. Nath, and A. Meliou, “Causality-guided adaptive interventional debugging,” in Proc. ACM SIGMOD Int. Conf. Management of Data, 2020, pp. 431–446.

[18] S. Galhotra, A. Fariha, R. Lourenço, J. Freire, A. Meliou, and D. Srivastava, “DataPrism: Exposing disconnect between data and systems,” in Proc. ACM SIGMOD Int. Conf. Management of Data, 2022, pp. 217–231.

[19] S. Grafberger, P. Groth, J. Stoyanovich, and S. Schelter, “Data distribution debugging in machine learning pipelines,” The VLDB Journal, vol. 31, no. 5, pp. 1103–1126, 2022.

[20] S. Grafberger, S. Guha, P. Groth, and S. Schelter, “mlwhatif: What if you could stop re-implementing your machine learning pipeline analyses over and over?,” Proc. VLDB Endow., vol. 16, no. 12, pp. 4002–4005, 2023.

[21] J. Zhu, G. Xu, F. Lorenzi, B. Glavic, and B. Salimi, “Stress-testing ML pipelines with adversarial data corruption,” Proc. VLDB Endow., vol. 18, no. 11, pp. 4668–4681, 2025.

[22] M. D. Ernst, J. H. Perkins, P. J. Guo, S. McCamant, C. Pacheco, M. S. Tschantz, and C. Xiao, "The Daikon system for dynamic detection of likely invariants," Science of Computer Programming, vol. 69, no. 1–3, pp. 35–45, 2007.

[23] V. Vovk, A. Gammerman, and G. Shafer, Algorithmic Learning in a Random World. New York, NY, USA: Springer, 2005.

[24] A. Paleyes and N. D. Lawrence, "Causal fault localisation in dataflow systems," in Proc. 3rd Workshop on Machine Learning and Systems (EuroMLSys), 2023.

[25] J. Leest, I. Gerostathopoulos, P. Lago, and C. Raibulet, "Tracing distribution shifts with causal system maps," arXiv:2510.23528, 2025.

[26] A. Ikram, S. Chakraborty, S. Mitra, S. Saini, S. Bagchi, and M. Kocaoglu, "Root cause analysis of failures in microservices through causal discovery," in Advances in Neural Information Processing Systems (NeurIPS), 2022.

[27] S. Jha, J. Rios, F. Bagehorn, L. Shwartz, and N. Abe, "Fault localization using interventional causal learning for cloud-native applications," in Proc. 54th Annu. IEEE/IFIP Int. Conf. Dependable Systems and Networks – Supplemental Volume (DSN-S), 2024, pp. 141–147.

[28] J. Li, B. B. Chu, I. F. Scheller, J. Gagneur, and M. H. Maathuis, "Root cause discovery via permutations and Cholesky decomposition," J. Royal Statistical Society Series B: Statistical Methodology, vol. 88, no. 3, pp. 779–798, 2026, doi: 10.1093/jrsssb/qkaf066.

[29] W. R. Orchard, N. Okati, S. H. Garrido Mejia, P. Blöbaum, and D. Janzing, "Root cause analysis of outliers with missing structural knowledge," in Advances in Neural Information Processing Systems (NeurIPS), 2025.

[30] S. Shankar and A. G. Parameswaran, "Towards observability for production machine learning pipelines," Proc. VLDB Endow., vol. 15, no. 13, pp. 4015–4022, 2022.

[31] H. Foidl, V. Golendukhina, R. Ramler, and M. Felderer, "Data pipeline quality: Influencing factors, root causes of data-related issues, and processing problem areas for developers," J. Systems and Software, vol. 207, 2024.

[32] N. Sambasivan, S. Kapania, H. Highfill, D. Akrong, P. Paritosh, and L. M. Aroyo, "'Everyone wants to do the model work, not the data work': Data cascades in high-stakes AI," in Proc. CHI Conf. Human Factors in Computing Systems, 2021.

[33] C. K. Chow, "On optimum recognition error and reject tradeoff," IEEE Transactions on Information Theory, vol. 16, no. 1, pp. 41–46, 1970.

[34] R. El-Yaniv and Y. Wiener, "On the foundations of noise-free selective classification," Journal of Machine Learning Research, vol. 11, pp. 1605–1641, 2010.

[35] U.S. Census Bureau, "American Community Survey Public Use Microdata Sample (PUMS), 1-year files 2014–2024." [Online]. Available: https://www.census.gov/programs-surveys/acs/microdata.html

[36] NYC Taxi and Limousine Commission, "TLC trip record data." [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page

[37] R. Misra, "News category dataset," arXiv:2209.11429, 2022.

[38] Divvy, "Divvy system data." [Online]. Available: https://divvybikes.com/system-data

[39] Consumer Financial Protection Bureau and Federal Financial Institutions Examination Council, "Home Mortgage Disclosure Act data: historic data (2007–2017) and HMDA Data Browser (2018 on)." [Online]. Available: https://www.consumerfinance.gov/data-research/hmda/historic-data/ and https://ffiec.cfpb.gov/data-browser/

[40] Bureau of Transportation Statistics, "Reporting Carrier On-Time Performance (1987–present)." [Online]. Available: https://www.transtats.bts.gov/

[41] Capital Bikeshare, “System data.” [Online]. Available: https://capitalbikeshare.com/system-data

[42] National Highway Traffic Safety Administration, “Fatality Analysis Reporting System (FARS).” [Online]. Available: https://www.nhtsa.gov/research-data/fatality-analysis-reporting-system-fars

[43] HM Land Registry, “Price Paid Data.” Contains HM Land Registry data © Crown copyright and database right, licensed under the Open Government Licence v3.0. [Online]. Available: https://www.gov.uk/government/statistical-data-sets/price-paid-data-downloads

[44] NOAA National Centers for Environmental Information, “Storm Events Database,” bulk CSV files of event details, fatalities and locations. [Online]. Available: https://www.ncei.noaa.gov/pub/data/swdi/stormevents/csvfiles/

[45] City of Chicago, “Traffic Crashes - Crashes,” “Traffic Crashes - People” and “Traffic Crashes - Vehicles,” Chicago Data Portal. [Online]. Available: https://data.cityofchicago.org/Transportation/Traffic-Crashes-Crashes/85ca-t3if

[46] The pandas development team, “What’s new in 3.0.0,” pandas documentation, 2026. [Online]. Available: https://pandas.pydata.org/docs/whatsnew/v3.0.0.html

[47] The pandas development team, “Copy-on-Write (CoW),” pandas user guide. [Online]. Available: https://pandas.pydata.org/docs/user_guide/copy_on_write.html

[48] pradipneupane, “BUG: pyarrow engine timestamp conversion fails when doing pd.read_csv,” pandas-dev/pandas issue #65308, GitHub, Apr. 2026. [Online]. Available: https://github.com/pandas-dev/pandas/issues/65308

[49] eddiebergman, “Basic Example with 2019 gives no key ‘RELP’ found,” socialfoundations/folktables issue #22, GitHub, Jun. 2022. [Online]. Available: https://github.com/socialfoundations/folktables/issues/22

[50] cruyffturn, “Postprocessing step np.nan_to_num,” socialfoundations/folktables issue #39, GitHub, Jan. 2024. [Online]. Available: https://github.com/socialfoundations/folktables/issues/39

[51] marklit, “Date/time value is out of range in the Uber 2014 data,” toddwschneider/nyc-taxi-data issue #3, GitHub, Feb. 2016. [Online]. Available: https://github.com/toddwschneider/nyc-taxi-data/issues/3

[52] HoneyTyagii, “OneHotEncoder: inverse_transform decodes unknown categories as the dropped category (not None) when drop + handle_unknown=‘ignore’,” scikit-learn/scikit-learn issue #34549, GitHub, Jul. 2026. [Online]. Available: https://github.com/scikit-learn/scikit-learn/issues/34549

[53] B. Gluss, “An optimum policy for detecting a fault in a complex system,” Operations Research, vol. 7, no. 4, pp. 468–477, 1959.

Appendix A: Proofs of the results of Section 4

Theorem 1. A monitoring-only diagnostic φ is a (possibly randomised) measurable function of observations whose joint law is identical under (D, σ, θ) and (D', σ, θ'), and the change logs are identical by assumption; hence the output law of φ is identical, and P[φ = defect] is the same under both. Its type-I error on the non-defective configuration plus its type-II error on the defective one is therefore 1 − P[φ = defect] + P[φ = defect] = 1. ∎

Corollary 1. (⇐) Let the corrupted law of (X, Y) be supported in Img × Ω_Y. Choose a measurable selection w(x) ∈ $\left(P_{\theta^*} \circ \sigma\right)^{-1}(x)$ for x ∈ Img and push the corrupted law forward through (x, y) ↦ (w(x), y) to obtain D'; the correct pipeline applied under D' produces the corrupted law. (⇒) Mass outside Img is impossible under every non-defective configuration; a support test that flags any record outside Img has no false alarms and misses the defect only if all n records fall in Img, which has probability $(1 - \varepsilon)^n$. ∎

Theorem 2. (a) With deterministic components and $\theta_c^* = \theta_c^0$, c is locally effective on its current inputs I_c iff $f_c^{\theta_c}(I_c) \neq f_c^{\theta_c^0}(I_c)$, which is exactly the comparison the rollback test makes on archived inputs. (b) If $f_c^{\theta_c}($ $I_c) = f_c^{\theta_c^0}(I_c)$, every downstream component receives identical inputs under both versions and, being deterministic, produces identical outputs; the final output is unchanged. (c) Replacing θ_c by $\theta_c^* = \theta_c^0$ changes only c's output, from $f_c^{\theta_c}(I_c)$ to $f_c^{\theta_c^0}(I_c)$; the final output under the replacement is the downstream subgraph D_c, at its current versions, applied to the rolled-back output (with all other inputs as archived), which is what the propagation test computes. c is effective iff that output differs from the current one. The converse of (b) fails exactly when D_c maps the two outputs to the same final output, e.g. when a downstream step drops or filters the records that differ. (d) If no component is forced, $\theta^* = \theta^0$, which agrees with the current versions outside Δ. Induction over the topological order of all components (an untested component outside Δ equals its specification and receives specification inputs): when c is tested, every upstream output already equals its specification's; if c's current version agrees with θ_c^0 on these inputs its output equals the specification's, and otherwise replacing it makes it so. The final output is therefore P_{θ^*}(R). Each rollback executes one component once and each propagation test one downstream subgraph. ∎

Theorem 3. Every re-execution of pipeline components takes raw records (of the current or of the reference period) as input, and archived outputs are such executions. The reference period is the same in both configurations, and in the current period the raw records and labels have the same law by assumption (σ^1#D = σ^0#D'' in (a), $\sigma^{1'}$#D = $\sigma^{1'}$#D' in (b)), so every statistic of any re-execution, and every monitoring statistic, has the same law in both configurations, and the argument of Theorem 1 applies. In (b), a certificate is a function of these statistics, so it takes the same value in law in both configurations and is wrong in the one where the adapter's correctness differs from its verdict. (a) is the special case $\sigma^{1'} = \sigma^0$. ∎

Proposition 1. Consequential defects are effective by definition. (a) A fixed c lies outside T, so it is not forced (accurate change record and lineage) and $\theta_c^* = \theta_c^0$. Its propagation test re-executes the downstream subgraph on $f_c^{\theta_c^0}(I_c) = f_c^{\theta_c^*}(I_c)$ with every other component at its current version, which is the final output under the replacement of θ_c by θ_c^*; it differs from the current output, so c is an effective defect (Theorem 2c). (b) Let c be a consequential defect. If c ∈ T it is unverified. Otherwise c is not forced, and being a defect it is changed ($f_c^{\theta_c} \neq f_c^{\theta_c^*} = f_c^{\theta_c^0}$, so $\theta_c \neq \theta_c^0$), hence listed in the complete change log. It is effective, hence locally effective (Theorem 2b), so its rollback differs (Theorem 2a) and its propagation test differs (Theorem 2c), unless a test cannot run, in which case c is unverified. (c) If T = ∅, no component is forced; an unchanged component then equals its specification and is not a defect; if every test runs, U = ∅, and by (a) and the argument of (b) a changed component is fixed iff it is an effective defect. The fix set is the set of effective defects, which contains the consequential ones. (d) Configuration 1 is (D, σ^1), where σ^1 is an encoding consistent with the published change under which the deployed version of c is a correct adaptation (it exists because c's deployed output is attainable by a correct adapter) and σ^1#D is the observed raw law. Configuration 2 is (D'', σ^0) with σ^0#D'' = σ^1#D, in which the published change leaves the encoding of c's columns unchanged. There $\theta_c^* = \theta_c^0 \neq \theta_c$, so c is a defect, consequential by the observed effect of its rollback. Both configurations share the raw-record law, the reference period, the

change log and the change record (hence T), and by Theorem 3 every statistic available to the rule has the same law in both. Fixing c is a false blame in the first; clearing it is a silent miss in the second. ∎

Appendix B: Side information for forced changes

Proposition 2 (test ordering). Under a single-fault prior p_c with test costs k_c, testing candidates in decreasing p_c/k_c minimises the expected cost of finding the faulty component (sequential fault search with test costs goes back to [53]).

Theorem 3 means that a forced change can only be judged with side information. Round 1 used one assumption, *world smoothness at published changes*: a component's output should not jump at a change much more than it moves between ordinary periods. Its weak point is that providers change layouts *when the world changes* (a new fee needs a new column). Two further kinds of evidence do not depend on the world staying still.

Assumption (I), invariant stability. An *invariant* of the semantic schema is a measurable g : $\Omega_S \to \{0, 1\}$ (1 = violated) whose violation rate under the correct adapter, $v_t = P_{D_t}[g(s(W)) = 1]$, is small and does not increase at the change by more than its largest increase over the m preceding transitions. (I′) is the weaker assumption that the m + 1 increases are exchangeable. Both are statements about the provider's *process* (accounting identities, business rules, timestamp order), not about the distribution of the world.

Lemma 1 (region invariance). Let g = 1[s $\notin$ Q] for a region Q $\subset \Omega_S$ that contains the support of s(W) under every world distribution the provider can produce in both periods. Then (I) holds for every world shift $D_{e-1} \to D_e$, however large, that keeps s(W) inside Q.

Theorem 4 (invariant evidence for forced changes). Let A_e be the deployed adapter at the change e, and among the records it gets wrong let V be those it moves out of the invariant region (the correct output satisfies g, the deployed one violates it) and V^- those it moves into it. (a) *Detection:* the violation rate of the deployed output is $\hat{v}_e = v_e + P(V) - P(V^-) \geq P(V)$. If the pipeline was correct in period e − 1 and P(V) exceeds v_{e-1} plus the largest increase over the m preceding transitions, the rule of (b) flags the deployed adapter for every world distribution satisfying (I). (b) *World-robustness:* for a correct adapter, the rule "flag if the violation-rate increase at the change exceeds the largest increase over the m preceding transitions" never flags under (I), in particular under Lemma 1, and flags with probability at most 1/(m+1) under (I′). (c) *Blind defects:* if P(V) = 0 and the deployed output is attainable by a correct adapter under some world distribution in $\mathcal{F}$ (Corollary 1), the invariant carries no evidence and Theorem 3 applies again.

Proposition 3 (excusing a jump). If an invariant determines a jumping column k from other columns z (support, sum and difference identities), the invariant is intact at the change, and the z columns are produced correctly, then the jump of k, whatever its size, is no evidence against the adapter. The proposition removes evidence; it does not certify the adapter.

Proposition 4 (history rule). Let the world-induced change statistics $J_1, \ldots, J_m$ of the m ordinary transitions before a change and J_e of the change transition be exchangeable for a correct pipeline. The rule "flag if $J_e > \max(J_1, \ldots, J_m)$" has false-flag probability at most 1/(m+1) per column; with L columns at most L/(m+1). Round 1 compared J_e with a single transition, for which no bound below 1/2 holds.

B.1 Validity conditions for re-execution evidence

Proposition 5 (superseded layouts carry no evidence of correctness). Let the source publish layouts σ_1 , …, σ_k at changes $e_1 < \ldots < e_k \le$ p. The correct adapter for period p need only satisfy $A_p^* \circ \sigma_k$ = s: for any adapter A that is correct on σ_k there is an adapter A' that equals A on the support of σ_k, fails on the support of σ_j outside that of σ_k (j < k), and is also correct. Hence no test whose outcome depends on how the adapter treats an earlier layout is a consistent test of its correctness at p; a forced component may only be judged at the latest published change that touches its columns.

Proposition 6 (validity of a rollback counterfactual). Theorem 2 and the effect test of DoD (Appendix C) use the reference version θ_c^0 as a known-good counterfactual. If θ_c^0 maps semantic values that the current version reads to *missing* (it cannot parse the new layout), its output lies outside the image of the correct pipeline whenever the field is always defined, and the effect test compares the current version with a broken counterfactual; a non-improvement is then no evidence that the current version is harmless. Such a rollback must be treated like one that fails.

Proposition 7 (introduced missingness is world-robust evidence). Let a semantic field k be always defined by the provider (a trip has a start time). This is a support invariant in the sense of Lemma 1. If a component's output has k missing on a fraction ρ > 0 of records while the inputs it reads k from are defined there, the component maps those records out of the invariant region; under (I) this happens with probability 0 for a correct component, whatever the world does. An increase of the missing rate of a component's output relative to the reference period that is not present in its inputs is therefore evidence of a defect that needs neither history nor smoothness.

Propositions 5–7 do not weaken Theorem 3: they remove ways in which a correct adapter can be *mistaken* for a defect (judged on layouts it no longer needs to read; judged against a rollback that cannot read the current layout), and add one form of evidence for defects that destroy values. A third condition needs no proposition: a history test may only use periods that the deployed versions can read.

B.2 Proofs

This subsection proves the results stated in this appendix, in the order in which they are stated.

Proposition 2. Let candidates i and j be adjacent in an order, with i first. Swapping them changes only the expected cost of the two tests: testing i first costs $k_i + (1 - p_i)\, k_j$ relative to the common remainder, testing j first $k_j + (1 - p_j)\, k_i$, and the first is no larger iff $p_i\, k_j \ge p_j\, k_i$, i.e. $p_i/k_i \ge p_j/k_j$. Any order that is not sorted by decreasing p/k therefore contains an adjacent pair whose swap does not increase the expected cost, and repeated swaps reach the sorted order. ∎

Lemma 1. $\nu_e = P_{D_e}[\mathrm{s(W)} \notin \mathrm{Q}] = 0 = \nu_{e-1}$, since Q contains the support of s(W) under both distributions; the increase is 0, which never exceeds the largest earlier increase. ∎

Theorem 4. (a) On records the deployed adapter gets right, the deployed and the correct outputs agree; on V the deployed output violates g and the correct one does not; on V^- the reverse; on the remaining wrong records both or neither violate it. Taking expectations gives $\hat{\nu}_e = \nu_e + \mathrm{P(V)} - \mathrm{P(V^-)}$, and $\mathrm{P(V^-)} \le \nu_e$ because the correct output violates g on V^-, so $\hat{\nu}_e \ge \mathrm{P(V)}$. With the pipeline correct in period e − 1, the deployed increase at the change is $\hat{\nu}_e - \nu_{e-1} \ge \mathrm{P(V)} - \nu_{e-1}$, which exceeds the largest earlier increase by

assumption, so the rule flags, whatever the world distribution. (b) Under a correct adapter the violation-rate increases of the m + 1 transitions are exchangeable by assumption, so the event increase is the strict maximum with probability at most 1/(m+1) (Proposition 4) under (I′); under (I) the increase at the change never exceeds the largest earlier one, so the rule never flags, and Lemma 1 gives (I). (c) If the deployed output is attainable by a correct adapter under some $D' \in \mathscr{F}$ (Corollary 1), the correct pipeline under D' produces the deployed output law, including the law of its violation indicators, so no function of the indicators separates the two configurations; P(V) = 0 is necessary for this, since records in V violate g with correct inputs. ∎

Proposition 3. On records satisfying g, k = h(z) for a fixed function h. If the z columns are produced correctly, the deployed k on those records is h of correct semantic values and hence attainable by a correct adapter under some world distribution (Corollary 1); a jump of k alone therefore carries no evidence against the adapter. It carries none for it either: a defect elsewhere in the adapter is not excluded. Order constraints do not determine k and cannot be used in this way. ∎

Proposition 4. Under exchangeability the rank of J_e among $J_1, \ldots, J_m, J_e$ is uniform over the m + 1 positions (ties only lower the probability of a strict maximum), so $P[J_e > max_i\ J_i] \leq 1/(m+1)$. The union bound gives L/(m+1) for L columns. Additional thresholds can only remove flags. ∎

Proposition 5. Correctness at p is the property $A \circ \sigma_k = s$ on the support of $\sigma_k(W)$. Given A correct at p, define A' = A on the support of σ_k and arbitrary (e.g. failing) on the support of $\sigma_j \setminus \text{supp}\ \sigma_k$. A' is also correct at p, but a test that depends on behaviour on σ_j distinguishes A from A'. Hence such a test rejects a correct adapter (A or A'), so its outcome is not a function of correctness at p. ∎

Proposition 6. If the field k is defined under every world distribution, the image of the correct pipeline contains only records with k defined. A rolled-back output with k missing on a positive fraction of records lies outside that image (Corollary 1) and is not the output of the correct pipeline for any world. An effect test that finds no improvement from the rollback therefore compares the current version with an output that is wrong for every world; the current version may be wrong as well (for example, wrong on other fields), so the test result does not bear on its correctness. ∎

Proposition 7. Let Q = {k defined}. By assumption the correct semantic output lies in Q for every world, so a correct component whose inputs carry k defined on a record outputs k defined on it. If the component's output has k missing on a fraction $\rho > 0$ of records whose inputs carry it, those records are mapped out of Q, which a correct component does with probability 0 under every world distribution (Lemma 1). An increase of the output's missing rate relative to the reference period that is absent in the inputs is therefore evidence against the component that does not depend on the world. ∎

Appendix C: The deciding versions DoD, DoD-2 and DoD-3

C.1 DoD (round 1)

- **Candidates.** Every changed component, and every unchanged component that depends, through column-level lineage, on a column touched by a published change and either reads the source or introduces drift at its node (a possible stale or latent defect). A changed component that depends on such a column is *treated as forced* (a proxy for forced in the sense of Section 3).

- **Ordering.** Candidates are tested in decreasing ratio of prior to measured execution time (Proposition 2). The prior combines the drift *introduced* at the component's node (drift of its output columns beyond that of the same columns at its inputs, from archived outputs at no cost), drift and validation evidence on the model inputs it feeds, and source coupling.
- **Tests.** *Rollback* re-executes the reference version on the component's current inputs and compares outputs exactly; for an unforced component a difference shows a locally effective defect (Theorem 2a), which in the benchmark cases was almost always also effective (Section 10). For a component treated as forced, an *effect* test propagates the rolled-back output to the deployed model; an improvement above $\tau = 0.003$ marks it harmful; otherwise, and whenever the reference version cannot run on the new inputs or the change alters the label or the set of records, a *boundary* test compares the component's output just before the published change with its output at the change, relative to the ordinary change between the two preceding periods. A jump above 3 times the ordinary change and above 0.05 in population stability index (PSI), new missing values, or lost records flag the component.
- **Action.** Fix every harmful or flagged component; retrain if the accuracy drop that remains after the fix is at least 0.01.

C.2 DoD-2 (round 2)

DoD-2 changes only the boundary test. (i) *History rule:* the introduced change at a published change must exceed every one of the four preceding ordinary transitions of the deployment history (and 3 times their median, and 0.05 PSI); this is Proposition 4 with $m = 4$, bound 1/5 per column. (ii) *Invariant evidence:* invariants of four templates, namely support ($k \neq 0 \Leftrightarrow z \in Z$), order ($a \leq b$), sum ($t = a + b$ (+ c)) and difference ($d = (b - a)/u$), are mined from the component's archived outputs in the history periods; one is kept if it holds on at least 98% of records in every history period and is violated at least five times as often after a random permutation of its first column. A violation increase of at least 0.01 at the change flags the component (Theorem 4a); a jump explained by an intact invariant is excused (Proposition 3).

C.3 DoD-3 (round 3)

DoD-3 implements Propositions 5–7 on top of DoD-2:

- **Latest-change rule (Proposition 5).** A source column is judged only at the latest published change, up to the current period, that touches it; a downstream component only at the latest change relevant to it. Failures to read earlier layouts are not evidence.
- **Valid counterfactuals (Proposition 6).** If the reference version leaves more than 1% more values missing than the current version on the columns where they differ, the rollback is treated as failed and the effect test is not used; conversely (a heuristic in the spirit of Proposition 7, which assumes the field is always defined), a current version that leaves more than 10% of values missing that the reference version reads is harmful.
- **Readable history only.** The smoothness test uses up to four historical transitions and needs three (bound 1/4 by Proposition 4) where the archive has them; where the archive itself is shorter it uses what there is, down to one transition, as DoD did. If periods exist but the deployed versions cannot read them, the method abstains from the smoothness test instead of treating the failure as a flag. Transitions into periods with another published change relevant to the component are not

ordinary and are left out of the history (the exchangeability assumption of Proposition 4). Free-text columns (more than 50 distinct values) are not compared.

- **Introduced missingness (Proposition 7).** Where no change could be judged otherwise, a component is flagged if it introduces at least 10 percentage points of missing values beyond its reference period while its inputs stay complete.

Ablations: *DoD-3 without latest-change rule* keeps only the history and text-column changes; *DoD-3 without invariants* drops invariant evidence. All thresholds were fixed on development data and frozen.

Appendix D: Rounds 1–3 in detail

This appendix reports rounds 1–3 as pre-registered: every hypothesis with its verdict (Table 13), the results per domain, and the analyses of the failures that motivated the next version.

D.1 Round 1: census, taxi and news (DoD)

Table 7. Round 1, test split (278 cases). Correct diagnosis on cases with a consequential defect; false blame over all test cases. The terra agents were run only on cases with a consequential defect.

Method	Correct, all (n=97)	News (n=49)	Taxi (n=48)	Silent (n=39)	False blame (all cases)
Retrain on alarm	0.00	0.00	0.00	0.00	0.00
Validation first	0.10	0.10	0.10	0.00	0.48
Observational attribution	0.15	0.12	0.19	0.28	0.67
Full rollback	0.03	0.04	0.02	0.05	0.30
Bisection	0.43	0.20	0.67	0.38	0.14
Pipeline-instance debugging	0.53	0.39	0.67	0.54	0.11
LLM agent, report (luna)	0.31	0.47	0.15	0.46	0.56
LLM agent, tools (luna)	0.49	0.61	0.38	0.56	0.32
LLM agent, report (terra)	0.25	0.33	0.17	0.31	–
LLM agent, tools (terra)	0.60	0.80	0.40	0.85	–
DoD without boundary test	0.48	0.18	0.79	0.46	0.14
DoD	**0.74**	**1.00**	**0.48**	**1.00**	**0.33**

H1 (monitoring cannot see silent defects): not supported, not disproved. Of the 97 test cases with a consequential defect, 39 were *validation-silent*: they passed every validation check on the model inputs, an observable stand-in for the image condition of Corollary 1. Retrain-on-alarm and validation-first diagnosed 0.00 and 0.00 of them, observational attribution 0.28. The LLM agent reading the same report reached 0.46 (above the bound of 0.35, below the disproof threshold of 0.60), mostly by blaming changed components, which the change log rather than monitoring offers.

H2 (interventions resolve them): not supported. DoD diagnosed 0.74 of consequential test defects against 0.60 for the best baseline (LLM agent, tools (terra)); the difference +0.14 (95% CI [+0.05, +0.25], Holm p = 0.008) fell short of the pre-registered minimum effect of 0.15. DoD was correct on 1.00 of news cases and 0.48 of taxi cases (Table 7, Fig. 3).

H3 (ordering): supported. Ordering tests by evidence per cost under a one-run budget reached 0.60 against 0.49 for pipeline order (difference +0.10, 95% CI [-0.02, +0.24]); DoD used 13.9 s of compute per case, more than pipeline-instance debugging (4.1 s). **H4 (fewer errors than the default responses): partly supported.** DoD missed fewer defects than retrain-on-alarm (-0.32) and validation-first (-0.23), blamed fewer healthy components than validation-first (-0.14), and its retrained models were less fragile:

those chosen by retrain-on-alarm lost 0.16 more accuracy once the pipeline was repaired (all Holm $p < 0.01$). Against retrain-on-alarm, which never blames a component, its false-blame rate was higher (+0.33).

H5 (the predicted limit): supported. On forced (T1) consequential defects DoD was correct in 0.95 of cases and its rollback-only ablation (DoD without boundary test) in 0.14 (n = 21). The smoothness assumption failed on taxi data at one event: all 93 taxi test cases with a false blame straddle April 2023, and 90 of them come from one boundary flag on the airport-fee column, which was renamed in the quarter the fee rose from $1.25 to $1.75. The world jumped exactly at the published change.

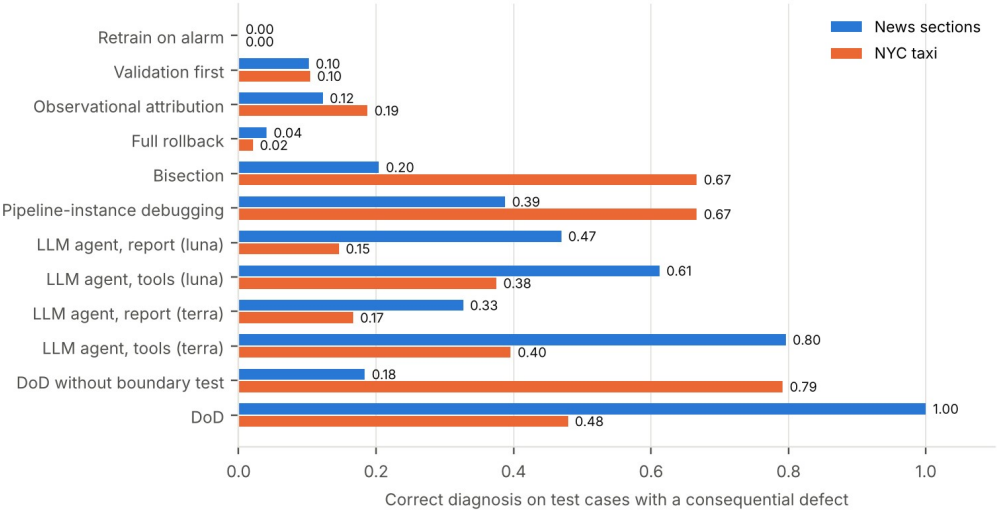


Fig. 3. Round 1: correct diagnosis on test cases with a consequential defect, by domain.

D.2 Round 2: three held-out domains (DoD-2)

Table 8. Round 2 (all cases are test cases). Correct diagnosis on cases with a consequential defect; false blame on all cases whose periods span a published change. Within the round's API allowance, the terra agents covered 49 of 146 cases of the planned every-third sample of cases with a consequential defect (the first in run order).

Method	Correct, all (n=435)	Bike CHI (n=228)	Mortgages (n=107)	Airlines (n=100)	Silent (n=40)	False blame at changes (n=617)
Retrain on alarm	0.00	0.00	0.00	0.00	0.00	0.00
Validation first	0.17	0.09	0.28	0.24	0.00	0.81
Observational attribution	0.42	0.28	0.46	0.68	0.72	0.72
Full rollback	0.08	0.02	0.05	0.24	0.12	0.02
Bisection	0.23	0.06	0.21	0.61	0.62	0.00
Pipeline-instance debugging	0.36	0.17	0.52	0.63	0.68	0.11
LLM agent, report (luna)	0.27	0.14	0.42	0.41	0.50	0.81
LLM agent, tools (luna)	0.41	0.23	0.54	0.69	0.70	0.66
LLM agent, report (terra)	0.37	0.58	0.41	0.20	0.67	–
LLM agent, tools (terra)	0.80	0.83	0.76	0.80	0.83	–
DoD without boundary test	0.49	0.20	0.62	1.00	0.95	0.71
DoD	**0.35**	**0.07**	**0.34**	**1.00**	**0.93**	**0.88**
DoD-2	**0.40**	**0.16**	**0.34**	**1.00**	**0.93**	**0.85**

H6–H8: H6 not supported; H7 not supported; H8 not supported. DoD-2 diagnosed 0.40 of consequential defects against 0.35 for DoD (H6: +0.05, Holm p = 0.37); its false-blame rate on cases spanning a published change was 0.85 against 0.88 (H7: -0.03, Holm p = 0.37); and it fell short of the best baseline, the tool-using terra agent, by 0.16 on the cases both were run on (H8, n = 49, 95% CI [-0.37, +0.02], Holm p = 0.37). Both DoD versions were perfect on the airline control domain (1.00) and failed where the source changed repeatedly: DoD-2 reached 0.16 (DoD: 0.07) on Chicago bike share and 0.34 on mortgages, with DoD-2 false blame of 0.86 and 0.65 on all cases of those domains (Fig. 4).

H9 (replication of H1): disproved. Retrain-on-alarm and validation-first again diagnosed 0.00 and 0.00 of the 40 silent consequential defects, but observational attribution reached 0.72, above the disproof threshold of 0.60. This does not contradict Theorem 1, which concerns diagnostics that see only the joint law of features and labels: observational attribution also reads the change log and the archived outputs of every component, and blames the changed component that introduces drift. When the defect is the only change that moves its node's output, that suffices; H1's operationalisation lumped this method with pure monitoring, and round 2 showed the distinction matters.

H10 (invariant evidence, descriptive). No mined invariant flagged a component in any consequential T1 case (0 of 179); DoD-2 without invariants made identical decisions. The history rule accounted for DoD-2's whole gain over DoD on T1 defects (0.24 vs 0.13).

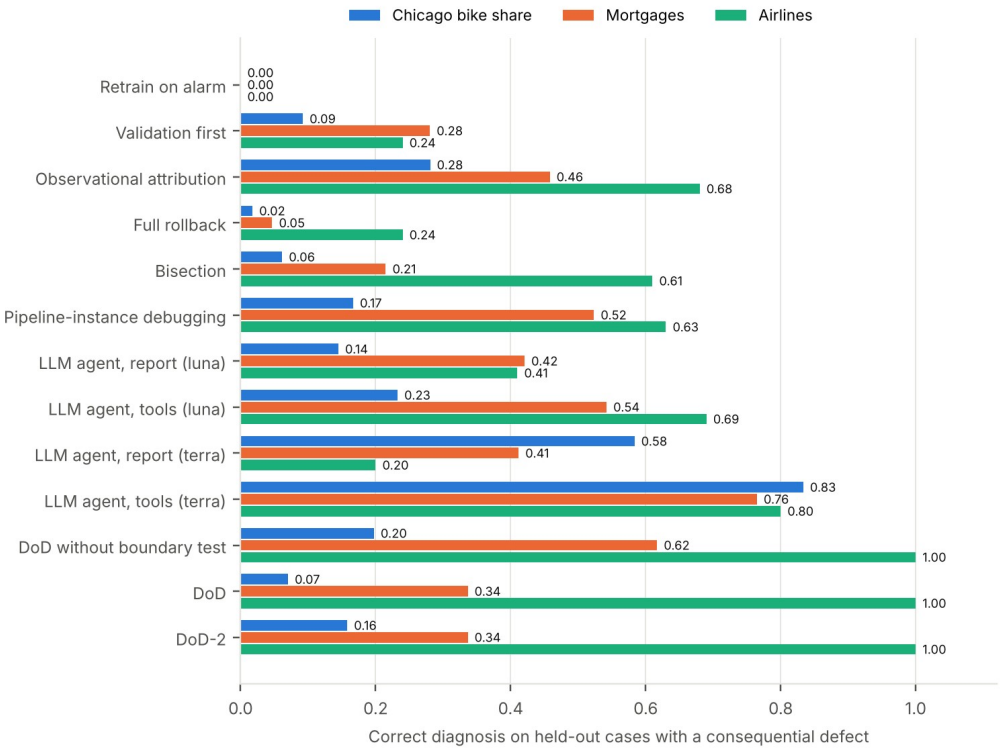


Fig. 4. Round 2: correct diagnosis on held-out cases with a consequential defect, by domain.

Analysis of the false blames (round-2 data, which is development data from round 3 on). I read the decisions behind every false blame. Three causes accounted for most of them, each a violation of a condition the tests had assumed without stating it: correct adapters were flagged for failing to read layouts the source had already replaced (the Chicago bike-share files changed at 10 dated points between 2017

and 2024; Proposition 5); the effect test compared a correct new adapter against a reference version that could not parse the new layout and therefore produced missing values (Proposition 6); and the boundary test treated the deployed versions' inability to read periods before a change as a flag ("history that cannot be read"). Keeping only the boundary flags raised at the latest published change that touches each flagged column (Proposition 5) lowers DoD's false-blame rate in the saved round-2 decisions from 0.65 to 0.42 on mortgages and from 0.91 to 0.65 on Chicago bike share. This is a post-hoc computation that motivated, but is not evidence for, DoD-3.

D.3 Round 3: three fresh held-out domains (DoD-3)

Table 9. Round 3 (all cases are test cases). Columns as in Table 8; compute is the measured re-execution time per case. The terra agents were run on the first 41 cases of the planned every-third sample of consequential cases (see text).

Method	Correct, all (n=373)	Bike DC (n=169)	Crashes (n=90)	Houses (n=114)	Silent (n=121)	False blame at changes (n=566)	Compute (s)
Retrain on alarm	0.00	0.00	0.00	0.00	0.00	0.00	0.0
Validation first	0.18	0.03	0.22	0.37	0.00	0.49	0.0
Observational attribution	0.46	0.30	0.82	0.41	0.81	0.64	0.0
Full rollback	0.07	0.04	0.07	0.11	0.10	0.12	1.5
Bisection	0.24	0.10	0.39	0.32	0.45	0.07	2.9
Pipeline-instance debugging	0.32	0.27	0.38	0.34	0.45	0.16	4.1
LLM agent, report (luna)	0.34	0.10	0.67	0.42	0.56	0.78	0.0
LLM agent, tools (luna)	0.47	0.24	0.66	0.66	0.61	0.42	6.9
LLM agent, report (terra)	0.51	0.53	0.60	0.36	0.48	–	–
LLM agent, tools (terra)	0.56	0.67	0.60	0.36	0.48	–	–
DoD without boundary test	0.42	0.20	0.51	0.68	0.61	0.25	10.4
DoD	**0.44**	**0.14**	**0.72**	**0.68**	**0.69**	**0.67**	**22.6**
DoD-2	**0.44**	**0.14**	**0.72**	**0.68**	**0.69**	**0.67**	**38.0**
DoD-3 without latest-change rule	0.45	0.11	0.80	0.68	0.71	0.67	40.1
DoD-3 without invariants	0.46	0.12	0.82	0.68	0.74	0.62	31.9
DoD-3	**0.46**	**0.12**	**0.82**	**0.68**	**0.74**	**0.62**	**44.0**

H11 (DoD-3 vs the best baseline): not supported. DoD-3 diagnosed 0.46 of consequential defects. The best baseline was LLM agent, tools (terra) (0.56 on the cases it was run on); the paired difference was -0.05 (95% CI [-0.33, +0.26], n = 41, Holm p = 0.882). The terra agents covered only 41 of the 125 cases of the planned every-third sample of consequential cases (the first in run order, within the round's API allowance), so their numbers rest on few cases. Among the other methods run on all cases, the best were LLM agent, tools (luna) (0.47) and observational attribution (0.46), on a par with DoD-3.

H12 (DoD-3 vs DoD): not supported. DoD reached 0.44 and DoD-2 0.44; the paired difference DoD-3 − DoD was +0.02 ([-0.02, +0.07], Holm p = 0.882).

H13 (false blame at published changes): supported. On the 566 cases whose periods span a published change (Fig. 6), DoD-3 blamed a healthy component in 0.62 of cases and DoD in 0.67 (difference -0.05, [-0.09, -0.01], Holm p = 0.009).

H14 (no change without published changes): supported. On the 243 cases without a published change in their period range, DoD-3 and DoD made the same diagnosis in 1.00 of cases.

Outcomes. The DoD versions rarely missed a consequential defect: DoD-3 missed one in 0.20 of consequential cases, against 0.54 for observational attribution, 0.64 for pipeline-instance debugging and 1.00 for retrain-on-alarm, and its recommended actions lost less accuracy relative to the best action than those of any non-DoD method run on all cases (regret on consequential cases 0.066 against 0.085). Regret scores an action under which the pipeline cannot run (typically one that leaves a crashing defect unfixed) as losing all accuracy; for every method most regret comes from such missed defects. The price is false blame: on all cases DoD-3 blamed a healthy component in 0.44 of cases.

Replication of the H1/H9 analysis (descriptive, not pre-registered for round 3). On the 121 silent consequential cases, retrain-on-alarm and validation-first again diagnosed 0.00 and 0.00; observational attribution, which also reads the change log and every component's archived output, reached 0.81, confirming the round-2 finding.

Invariants (the H10 analysis, descriptive). Mined invariants flagged a component in 3 of 146 consequential T1 cases, and DoD-3 without invariants made identical decisions on every case. Across rounds 2 and 3 the invariant evidence of Theorem 4 almost never applied: the defects of my catalogs rarely move records out of a region that the four templates describe (Theorem 4c).

Ablations. Without the latest-change rule, DoD-3 was correct on 0.45 of consequential cases (paired difference -0.01) but missed fewer defects (0.08) and had the lowest regret on consequential cases of all methods (0.024): judging a component only at the latest change that touches it removes false blame on superseded layouts, but also removes flags that, in my catalogs, coincided with stale adapters at earlier changes.

Where DoD-3 still failed (post hoc). The domains differ sharply (Fig. 5). On fatal crashes, DoD-3 was correct on 0.82 of consequential cases (DoD: 0.72), and its false-blame rate fell from 0.16 to 0.05: DoD had blamed the adapter for failing to read the byte-order-marked header of 2021–2022, a layout the source had already replaced; this is exactly the case of Proposition 5. On house sales every DoD version reached about two thirds, and all 97 of DoD-3's false blames of the cleaning step came from one flag: at the 2013 coverage change, the correct pipeline *intends* to drop the newly published non-standard sales, and the lost records look like a defect. On Washington bike share no method in Table 9 that was run on all cases exceeded 0.30 (the terra agents reached higher values on the 15 early cases they were run on). 198 false blames came from trip durations jumping at the 2020-Q2 layout change (early in the pandemic, when trips became longer and casual riders' share of trips rose sharply), and 130 from the heuristic of DoD-3 (after Proposition 7) that a current version leaving values missing is harmful: in the new layout, dockless trips legitimately have no station, so the correct station join leaves their round-trip indicator undefined, where the old version compared two blank station ids and called the trip a round trip. Each failure is a forced change whose correctness depends on what the new encoding means (a coincident world change, an intended change of population, and values that are missing by design). These are situations in which, by Theorem 3, no re-execution can certify the adapter, and in which the extra assumptions I added (smoothness, retention, Proposition 7's 'always defined') were false.

Development domains (in-sample). Table 10 reports DoD-3 on systematic samples of the round-1 and round-2 cases, which were used to design it; these numbers show what the changes fixed on the data they were designed on and are not evidence of generalisation.

Table 10. DoD-3 on the development domains (rounds 1–2 data; in-sample): correct diagnosis on consequential cases and false blame on all sampled cases. DoD-2 was run only on the round-2 domains.

Domain	Consequential cases	DoD	DoD-2	DoD-3	False blame DoD	False blame DoD-3
Census	46	0.96	–	0.96	0.01	0.01
Taxi	36	0.75	–	0.94	0.38	0.02
News	67	1.00	–	0.97	0.00	0.00
Bike (CHI)	53	0.09	0.21	0.19	0.90	0.53
Mortgages	58	0.33	0.33	0.50	0.66	0.49
Airlines	20	1.00	1.00	1.00	0.00	0.00

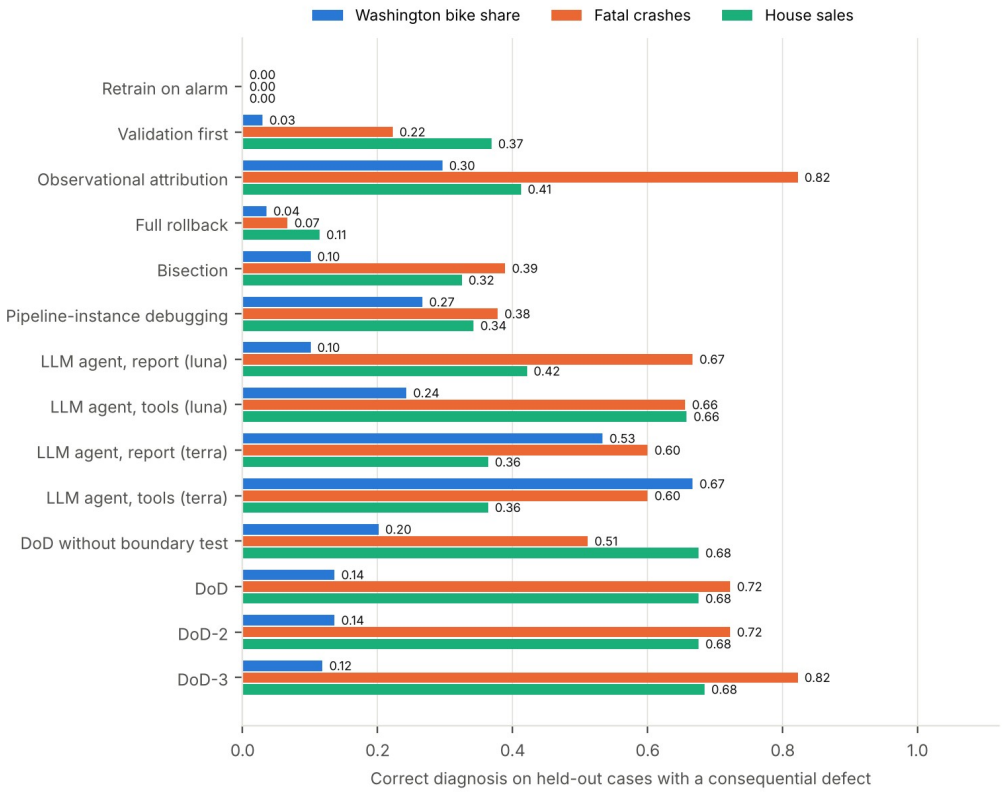


Fig. 5. Round 3: correct diagnosis on held-out cases with a consequential defect, by domain.

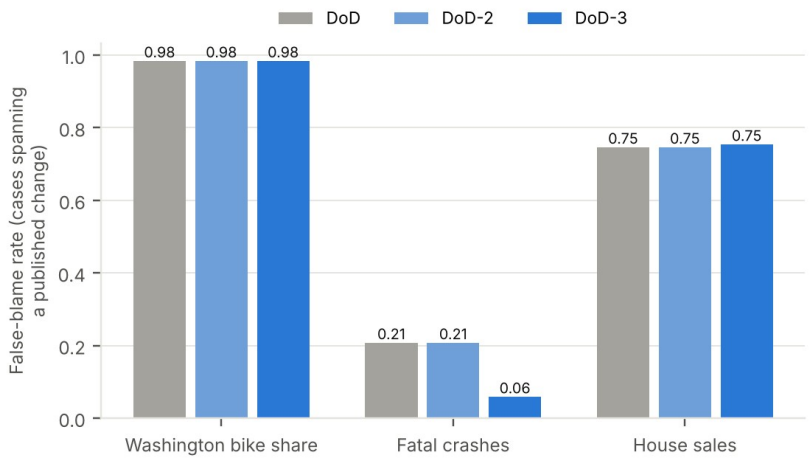


Fig. 6. Round 3: false-blame rate on cases whose periods span a published change.

Appendix E: Published changes and defect catalog

Table 11. Dated changes of the published files, found by profiling the raw files of every period (from each domain's first reference period on).

Domain	Period	Published change
Census	2017	SERIALNO format: 13-digit year-prefixed id
Census	2018	SERIALNO alphanumeric (GQ/HU); OCCP 2018 code list
Census	2019	RELP->RELSHIPP, WKW->WKWN, JWTR->JWTRNS
Census	2023	ST->STATE
Taxi	2018-07	passenger_count, RatecodeID published as double
Taxi	2019-01	congestion_surcharge populated
Taxi	2021-01	airport_fee populated
Taxi	2023-04	airport_fee renamed Airport_fee; VendorID/LocationIDs int32
Taxi	2025-01	cbd_congestion_fee column added
Taxi	2026-07	request_source column added
News	2014	new category values: ['ARTS', 'COLLEGE', 'EDUCATION', 'FIFTY', 'GOOD NEWS', 'GREEN', 'HEALTHY LIVING', 'LATINO VOICES', 'MEDIA', 'PARENTS', 'POLITICS', 'RELIGION', 'STYLE', 'TASTE', 'WEIRD NEWS', 'WOMEN', 'WORLDPOST']
News	2015	new category values: ['ARTS & CULTURE', 'THE WORLDPOST']
News	2017	new category values: ['WORLD NEWS']
News	2018	new category values: ['U.S. NEWS']
Bike (CHI)	2017-Q1	starttime/stoptime renamed start_time/end_time
Bike (CHI)	2017-Q4	timestamps published without seconds again
Bike (CHI)	2018-Q1	long-form column headers; ISO timestamps; trip duration with thousands separators
Bike (CHI)	2018-Q2	short column headers again
Bike (CHI)	2019-Q2	long-form column headers again
Bike (CHI)	2019-Q3	short column headers again
Bike (CHI)	2020-Q1	new layout: ride_id, rideable_type, started_at/ended_at, member_casual, coordinates; trip duration, gender and birth year removed
Bike (CHI)	2020-Q3	blank station ids (dockless trips); electric bikes
Bike (CHI)	2020-Q4	alphanumeric station ids
Bike (CHI)	2024-Q2	timestamps with fractional seconds
Mortgages	2014	missing values published as blanks instead of 'NA'
Mortgages	2015	missing values published as 'NA' again

Domain	Period	Published change
Mortgages	2017	numbers published without zero padding
Mortgages	2018	record format rebuilt (Regulation C): loan amount in dollars; loan purpose, occupancy and property recoded; state as postal code, county as 5-digit FIPS; tract fields renamed; open-end, reverse-mortgage and business-purpose flags added
Mortgages	2020	tract income percentage published with decimals
Mortgages	2024	tract income percentage published as integers again
Bike (DC)	2020-Q2	new layout: ride_id, rideable_type, started_at/ended_at, member_casual, coordinates; duration and bike number removed; timestamps partly with fractional seconds
Bike (DC)	2020-Q3	blank station ids (dockless trips); whole-second timestamps
Bike (DC)	2020-Q4	rideable_type code ‘classic_bike’ introduced next to ‘docked_bike’
Bike (DC)	2023-Q4	rideable_type code ‘docked_bike’ retired
Bike (DC)	2024-Q2	timestamps with fractional seconds
FARS	2015	ROAD_FNC replaced by RUR_URB and FUNC_SYS (new code list); label columns added
FARS	2016	county and city name columns added
FARS	2018	REL_ROAD code 12 (pedestrian refuge or traffic island) added
FARS	2020	WEATHER1, WEATHER2 and CF1-CF3 removed
FARS	2020	TYP_INT code 11 (other intersection type) added
FARS	2021	first header published with a byte-order mark (STATE)
FARS	2023	byte-order mark removed
FARS	2023	ROUTE (route signing) code list revised: new codes 0, 10-13, 95, 96, 99; frontage-road, ‘other’ and ‘unknown’ codes retired
Houses	2013	category B transactions (non-standard sales) and property type ‘Other’ published
Storms	2006	time-zone codes with UTC offset appended; source codes republished in title case with new names; damage amounts partly with two decimals
Storms	2006	indirect fatalities published; fatality-location codes extended
Storms	2006	only events with coordinates listed; several points per event
Storms	2007	old time-zone and source codes retired; tornado ratings on the Enhanced Fujita scale; ‘0.00K’ for no damage; ‘Volcanic Ash’ renamed ‘Volcanic Ashfall’; zone type ‘M’ retired
Storms	2008	reporting-source codes added
Storms	2012	reporting-source codes added
Storms	2014	reporting-source codes added
Storms	2015	reporting-source code added
Storms	2016	tornado rating ‘EFU’ (unrated) added
Storms	2021	daylight-time zone codes added
Crashes (CHI)	2019	revised report codes: intersection types in trafficway_type; codes added to traffic_control_device, first_crash_type, weather_condition and contributory causes
Crashes (CHI)	2019	detailed restraint codes in safety_equipment; pedestrian location and physical condition codes added
Crashes (CHI)	2019	first_contact_point code distribution shifts; vehicle type, maneuver and unit type codes added
Crashes (CHI)	2020	first_contact_point corner and side codes replace FRONT-LEFT, FRONT-RIGHT, REAR-LEFT, REAR-RIGHT and ROOF
Crashes (CHI)	2020	speeding cause codes retired
Crashes (CHI)	2023	‘CHILD RESTRAINT USED’ and ‘HELMET USED’ retired

Table 12. Defect types (benign decoys omitted). T1: tied to a published change; T2: real library-upgrade behaviour; T3: documented incident class; T3r: replay of a public incident. The last column names the documented behaviour or incident a defect replays or is modelled on.

Domain	Tier	Component	Defect	Documented source
Census	T2	person_clean	pandas 3 upgrade: chained-assignment fill becomes a no-op	–
Census	T3	features	income adjustment factor dropped (stale reference)	–

Domain	Tier	Component	Defect	Documented source
Census	T3	features	hours per week reported per month (unit change)	–
Census	T3	features	vehicles feature stuck at 0 for Texas (upstream default)	–
Census	T3	encode	categorical codes re-fitted at serving time (training/serving skew)	–
Census	T1	join_household	alphanumeric SERIALNO ‘fixed’ by stripping letters (GQ/HU ids collide)	–
Census	T1	join_household	join keys still parsed as numbers after the id format change	–
Census	T1	occ_group	occupation crosswalk not refreshed for 2018 codes	–
Census	T1	person_adapter	RELSHIPP codes shifted by -20 instead of the official crosswalk	–
Census	T1	person_adapter	WKWN binned into WKW categories in reverse order	–
Census	T1	person_adapter	WKWN weeks passed through as WKW categories	–
Census	T1	person_adapter	JWTRNS used as JWTR without recoding (subway/rail/streetcar codes permuted)	–
Census	T1	person_adapter	missing ST column filled with the most common training state	–
Taxi	T2	time_features	pandas 2.2 / pyarrow 25 upgrade: parquet timestamps keep microsecond unit; epoch math via astype(int64)	–
Taxi	T2	time_features	pandas 3 upgrade: epoch via astype(int64) changes resolution	–
Taxi	T2	clean	pandas 3 upgrade: chained-assignment cleaning becomes a no-op	–
Taxi	T3	time_features	timestamps shifted as if converted from UTC	–
Taxi	T3	zone_join	dropoff zone joined on the pickup id (copy-paste key bug)	–
Taxi	T3	fare_features	trip distance converted to km (unit change)	–
Taxi	T3	encode	categorical codes re-fitted at serving time (training/serving skew)	–
Taxi	T1	trip_adapter	rate code key built from the published dtype (‘1.0’ after the double change)	–
Taxi	T1	trip_adapter	case-sensitive lookup misses the renamed Airport_fee column (filled with 0)	–
Taxi	T1	trip_adapter	new congestion fee not separated; lands in other_charges	–
News	T3	text_norm	lower-casing step dropped	–
News	T3	text_norm	description dropped by a field-mapping bug	–
News	T3	text_norm	UTF-8 text decoded as Latin-1 upstream	–
News	T3	vectorize	vocabulary re-fitted at serving time (training/serving skew)	–
News	T1	label_map	section harmonisation table not refreshed for new section names	–
Bike (CHI)	T3r	time_features	month/day order swapped when parsing M/D/Y timestamps	nyc-taxi-data issue #3
Bike (CHI)	T3	station_join	end coordinates looked up with the start station id (copy-paste key bug)	–
Bike (CHI)	T2	clean	pandas 3 upgrade: chained assignment blanking round-trip distances becomes a no-op	pandas 3.0 release notes (copy-on-write)
Bike (CHI)	T3	encode	categorical codes re-fitted at serving time (training/serving skew)	–

Domain	Tier	Component	Defect	Documented source
Bike (CHI)	T3r	encode	unseen categories encoded as the reference category	scikit-learn issue #34549
Bike (CHI)	T1	time_features	timestamps parsed with the previous published format	–
Bike (CHI)	T1	time_features	trip duration with thousands separators parsed without removing them	–
Bike (CHI)	T1	trip_adapter	user type mapped with the Subscriber/Customer rule: every member/casual value becomes casual	–
Bike (CHI)	T1	station_join	station coordinates still taken from the 2017 station table instead of the records	–
Bike (CHI)	T1	station_join	blank station ids of dockless trips compared as equal: counted as round trips	–
Mortgages	T2	clean	pandas 3 upgrade: chained-assignment missing-income sentinel becomes a no-op	pandas 3.0 release notes (copy-on-write)
Mortgages	T3r	clean	np.nan_to_num(x, -1): missing income becomes 0 instead of -1	folktables issue #39
Mortgages	T3	features	loan-to-income ratio computed upside down	–
Mortgages	T3	encode	categorical codes re-fitted at serving time (training/serving skew)	–
Mortgages	T3r	encode	unseen categories encoded as the reference category	scikit-learn issue #34549
Mortgages	T1	lar_adapter	new adapter keeps the loan amount in dollars (was thousands)	–
Mortgages	T1	lar_adapter	new loan-purpose codes 31/32/5 passed through instead of recoded (rename without recode)	folktables issue #22
Mortgages	T1	lar_adapter	occupancy type read as the old owner-occupancy code (investment property becomes ‘not applicable’)	–
Mortgages	T1	lar_adapter	county key built the old way (state + county code) from the new 5-digit county code	–
Mortgages	T1	lar_adapter	new open-end / reverse / business-purpose flags ignored: products outside the old scope enter the data	–
Airlines	T2	time_features	pandas 3 upgrade: day of year via astype(int64) changes resolution	pandas issue #65308; pandas 3.0 release notes (datetime resolution inference)
Airlines	T3	time_features	scheduled hhmm clock times read as minutes	–
Airlines	T2	clean	pandas 3 upgrade: chained-assignment overnight arrival fix becomes a no-op	pandas 3.0 release notes (copy-on-write)
Airlines	T3	clean	label recomputed as delay > 15 instead of the published >= 15 flag	–
Airlines	T3	clean	diverted flights kept (scope filter bug)	–
Airlines	T3	airport_join	destination size joined on the origin airport id (copy-paste key bug)	–
Airlines	T3	airport_join	distance converted to km (unit change)	–
Airlines	T3	encode	categorical codes re-fitted at serving time (training/serving skew)	–
Airlines	T3r	encode	unseen categories encoded as the reference category	scikit-learn issue #34549
Bike (DC)	T3r	time_features	day-first timestamp parsing	nyc-taxi-data issue #3
Bike (DC)	T3	station_join	end coordinates looked up with the start station id (copy-paste key bug)	–
Bike (DC)	T2	clean	pandas 3 upgrade: chained assignment blanking round-trip distances becomes a no-op	pandas 3.0 release notes (copy-on-write)
Bike (DC)	T3	encode	categorical codes re-fitted at serving time (training/serving skew)	–

Domain	Tier	Component	Defect	Documented source
Bike (DC)	T3r	encode	unseen categories encoded as the reference category	scikit-learn issue #34549
Bike (DC)	T1	trip_adapter	user type mapped with the Member/Casual rule: every lowercase member becomes casual	–
Bike (DC)	T1	station_join	station coordinates still taken from the station table instead of the records	–
Bike (DC)	T1	time_features	timestamps parsed with the previous published format (fractional seconds unparsed)	–
Bike (DC)	T1	station_join	blank station ids of dockless trips compared as equal: counted as round trips	–
FARS	T2	clean	pandas 3 upgrade: chained-assignment unknown-code handling becomes a no-op	pandas 3.0 release notes (copy-on-write)
FARS	T3	features	weekend from a 0-based weekday convention (FARS uses 1 = Sunday)	–
FARS	T3	encode	categorical codes re-fitted at serving time (training/serving skew)	–
FARS	T3	encode	state key zero-padded while the encoder artifact holds unpadded codes	–
FARS	T3r	encode	unseen categories encoded as the reference category	scikit-learn issue #34549
FARS	T1	crash_adapter	rural/urban and road class still read from the removed ROAD_FNC field	–
FARS	T1	crash_adapter	FUNC_SYS codes mapped with the old ROAD_FNC table (rename without recode)	folktables issue #22
FARS	T1	crash_adapter	weather still read from the removed WEATHER1 field	–
FARS	T1	crash_adapter	state read from ‘STATE’ while the header carries a byte-order mark	–
FARS	T1	crash_adapter	route-signing codes read with the pre-2023 code list (no crosswalk)	–
Houses	T2	time_features	pandas 3 upgrade: day of year via astype(int64) changes resolution	pandas issue #65308
Houses	T3	time_features	day of week shifted to a 1-based convention	–
Houses	T2	clean	pandas 3 upgrade: chained-assignment log price becomes a no-op	pandas 3.0 release notes (copy-on-write)
Houses	T3	geo	postcode district used where the postcode area is expected	–
Houses	T3	encode	categorical codes re-fitted at serving time (training/serving skew)	–
Houses	T3r	encode	unseen categories encoded as the reference category	scikit-learn issue #34549
Houses	T1	clean	category B (non-standard) sales not excluded after the 2013 coverage change	–
Storms	T2	times	pandas 3 upgrade: duration via astype(int64) changes resolution	pandas issue #65308
Storms	T2	clean	pandas 3 upgrade: chained-assignment wind speed and hail size become no-ops	pandas 3.0 release notes (copy-on-write)
Storms	T3	times	event duration reported in minutes where hours are expected	–
Storms	T3	loc_agg	location points aggregated by episode instead of by event	–
Storms	T3	geo	latitude and longitude swapped in the grid cell	–
Storms	T3	clean	wind speed converted from knots to mph where knots are expected	–
Storms	T3	encode	categorical codes re-fitted at serving time (training/serving skew)	–

Domain	Tier	Component	Defect	Documented source
Storms	T3r	encode	unseen categories encoded as the reference category	scikit-learn issue #34549
Storms	T1	times	time-zone table not refreshed for codes with the UTC offset appended	–
Storms	T1	times	appended offset stripped and the old table used (Alaska and Guam codes lost)	–
Storms	T1	sources	source crosswalk not refreshed for the title-case code list	–
Storms	T1	sources	new source codes upper-cased and looked up in the old list (renamed codes lost)	–
Storms	T1	fatal_agg	indirect fatalities counted after they were added to the fatality file	–
Storms	T1	scale	tornado ratings still read on the Fujita scale (EF ratings unmapped)	–
Storms	T1	scale	unrated tornadoes (‘EFU’) coded as the lowest rating	–
Storms	T1	times	time-zone table not refreshed for the daylight-time codes	–
Crashes (CHI)	T2	times	pandas 3 upgrade: delay via astype(int64) changes resolution	pandas issue #65308
Crashes (CHI)	T2	clean	pandas 3 upgrade: chained-assignment indicator flags become no-ops	pandas 3.0 release notes (copy-on-write)
Crashes (CHI)	T3	times	notification delay reported in minutes where hours are expected	–
Crashes (CHI)	T3	person_agg	missing driver ages filled with 0 before averaging	–
Crashes (CHI)	T3	person_agg	cyclists counted as pedestrians	–
Crashes (CHI)	T3	geo	latitude and longitude swapped in the grid cell	–
Crashes (CHI)	T3	encode	categorical codes re-fitted at serving time (training/serving skew)	–
Crashes (CHI)	T3r	encode	unseen categories encoded as the reference category	scikit-learn issue #34549
Crashes (CHI)	T1	road	trafficway crosswalk not refreshed for the intersection codes	–
Crashes (CHI)	T1	road	new trafficway codes sent to the ‘other’ group instead of being mapped	–
Crashes (CHI)	T1	person_agg	restraint crosswalk not refreshed for the detailed restraint codes	–
Crashes (CHI)	T1	vehicle_agg	contact-point crosswalk not refreshed for the corner and side codes	–
Crashes (CHI)	T1	vehicle_agg	‘-CORNER’ suffix stripped and the old crosswalk used (side-front/rear and top codes lost)	–

Appendix F: Pre-registered hypotheses

Table 13. Hypotheses of the four rounds and their verdicts, as reported in Section 8 and Section 9 (details: Appendix D). Primary endpoint: diagnosis correctness on cases with a consequential defect.

	Round	Pre-registered hypothesis (abridged from the frozen protocol)	Verdict
H1	1	Monitoring-only methods (retrain on alarm, validation first, observational attribution, LLM report) reach at most 0.35 correctness on validation-silent consequential defects; disproved if any reaches 0.60.	not supported, not disproved

	Round	Pre-registered hypothesis (abridged from the frozen protocol)	Verdict
H2	1	DoD exceeds the best non-oracle baseline on the primary endpoint by at least 0.15, with the 95% paired cluster-bootstrap interval excluding 0.	not supported
H3	1	Under a one-run budget, ratio ordering is at least as correct as topological ordering.	supported
H4	1	DoD has lower missed-defect and false-blame rates than retrain on alarm and validation first, and lower fragility than retrain on alarm.	partly supported
H5	1	On source-coupled (T1) defects, DoD without boundary test is less correct than DoD.	supported
H6	2	DoD-2 is more correct than DoD (paired difference > 0, Holm $p < 0.05$).	not supported
H7	2	On cases spanning a published change, DoD-2 has a lower false-blame rate than DoD (Holm $p < 0.05$).	not supported
H8	2	DoD-2 exceeds the best non-DoD baseline by at least 0.15, interval excluding 0 (replication of H2).	not supported
H9	2	Replication of H1 on the round-2 domains.	disproved
H10	2	Descriptive: DoD-2 correctness on T1 defects with and without a flagging invariant.	descriptive
H11	3	DoD-3 exceeds the best non-DoD baseline (Holm $p < 0.05$; minimum effect of interest 0.10).	not supported
H12	3	DoD-3 exceeds DoD on the primary endpoint (Holm $p < 0.05$).	not supported
H13	3	On cases spanning a published change, DoD-3 has a lower false-blame rate than DoD (Holm $p < 0.05$).	supported
H14	3	Without a published change, DoD-3 and DoD make the same diagnosis in at least 95% of cases.	supported
H15	4	DoD-4's false-blame rate is at most 0.02, and on cases spanning a published change lower than DoD-3's (Holm $p < 0.05$).	supported
H16	4	DoD-4's silent-miss rate on cases with a consequential defect is at most 0.02.	supported
H17	4	Where DoD-4 decides, its diagnosis is correct in at least 95% (all decided cases and decided cases with a consequential defect).	supported
H18	4	On consequential cases without a published change, DoD-4 is correct in at least 95%.	supported
H19	4	Descriptive: how well DoD-3's evidence labels rank DoD-4's unverified components.	descriptive

Appendix G: Reproducibility and cost

Table 14. API usage of all LLM calls in the four rounds, from the cost ledger. All responses are cached; the study replays without API keys.

Model	Calls	Input tokens	Output tokens	USD
gpt-5.6-luna	15723	39,121,684	2,927,225	6.77
gpt-5.6-terra	1355	3,096,828	269,485	5.73

Round 1 protocol digest: 8eab1e0e42e9d2c90c99d95c45dc7df974ddf26ffab567ec97d95398875940d8 over 30 files.

Round 2 protocol digest: e50ee051bbc5b3ea3bd787b71f5b38c559df566c3eadd9cd46bdf2286313fb8d over 25 files.

Round 3 protocol digest: fb3229bdedc2f1eac54b786daeeb2efb9100d43283493a4c9772551d185a80a5 over 33 files.

Round 4 protocol digest: 9543f4c5d4e8f8c295df1bc291cde64141825d33040d8f9a97079f69cc51a100 over 36 files.

Pipelines ran on a 16-core laptop CPU; no GPU was used.

Every number in the text and tables can be recomputed from the saved records; the released index maps each number of the build-script version of the paper to the file it was computed from.